كلية هندسة الذكاء الاصطناعي
FACULTY OF ARTIFICIAL INTELLIGENCE ENGINEERING

Academic Community Platform (UniSphere)

A Senior project 2 report - submitted to complete the requirements for obtaining a bachelor's degree in Artificial Intelligence Engineering - Software Engineering and Intelligent Information Systems.

Prepared by

Sara Eyad Ata

Supervised by

Dr. Eng. Akram Massouh

August 2026

SUPERVISION CERTIFICATION

I Certify that the preparation of this project entitled

[Academic Community Platform (UniSphere)]

Prepared by

[Sara Eyad Ata]

was made under my supervision at Faculty of Artificial Intelligence Engineering

in partial Fulfillment of the Requirements for the Degree of **B.Sc.** in the Software Engineering and Intelligent Information Systems.

Name: …….

Signature: …….

Date: …….

نتقدم بخالص الشكر وعظيم الامتنان لكل من ساهم في إنجاز هذا العمل ودعمه علمياً وأكاديمياً، ولكل من كان له أثر، مباشر أو غير مباشر، في الوصول بهذا المشروع إلى صورته النهائية.

نتوجه بجزيل الشكر والتقدير إلى **الدكتور أكرم مسوح**، المشرف على هذا المشروع، لما قدمه من إشراف علمي متميز، وتوجيهات أكاديمية قيّمة، ودعم مستمر طوال مراحل تنفيذ المشروع. لقد كان لخبرته، وحرصه الدائم على تحفيزنا وتشجيعنا، الأثر الكبير في تعزيز ثقتنا بأنفسنا، ودفعنا نحو تقديم أفضل ما لدينا، وإخراج هذا المشروع بالصورة العلمية اللائقة.

كما نتقدم بخالص الشكر والتقدير إلى **أساتذتنا وأعضاء الهيئة التدريسية في الكلية**، الذين كان لعلمهم وجهودهم وتوجيهاتهم طوال سنوات الدراسة الدور الأساسي في بناء معارفنا العلمية والعملية، وإعدادنا لخوض هذه المرحلة بثقة وكفاءة.

وفي الختام، نتقدم بجزيل الشكر والامتنان إلى **جامعتنا وكليتنا** التي أتاحت لنا البيئة العلمية والتجربة الأكاديمية التي مكّنتنا من توظيف ما تعلمناه في هذا المشروع، وتحويل المعرفة النظرية إلى تطبيق عملي يعكس ما اكتسبناه خلال سنوات الدراسة.

والحمد لله أولًا وآخرًا، الذي بنعمته تتم الصالحات، وبفضله وتوفيقه وصلنا إلى هذه اللحظة، وأتممنا هذا العمل بعد رحلة من التعلم والاجتهاد والمثابرة.

المهندسة ساره اياد عطا

إهداء

إلى الذين كانوا لي وطنًا حين ضاق الطريق،

وإلى الذين آمنوا بي قبل أن أتعلم كيف أؤمن بنفسي...

إلى **أبي،**

إلى الرجل الذي علّمني أن الطريق مهما طال، فإن الوصول يستحق الصبر، وأن الطموح لا يعرف سقفًا.

كنتَ دائمًا سندي وفخري، ومنك تعلمت أن النجاح لا يُقاس بما نصل إليه فقط، بل بما نصبح عليه في الطريق.

إلى **أمي،**

إلى الدعاء الذي لم أسمعه دائمًا، لكنني رأيت أثره في كل خطوة. إلى القلب الذي أحبني دون شروط، وانتظر نجاحي وكأنه نجاحه. لكِ أهدي ثمرة سنواتي، فكل إنجاز أحققه يحمل بين سطوره شيئًا من تعبك وحبك ودعائك.

إلى **إخوتي،**

إلى من كانوا معي في أيام الفرح، وفي لحظات التعب، وفي كل مرحلة صنعت مني ما أنا عليه اليوم.

وجودكم في حياتي من أجمل النعم التي أحمد الله عليها

إلى **المهندس عبدالرحمن الحمود،**

إلى رفيق الدرب الذي جمعني به الطموح، وتشاركت معه الكثير من تفاصيل هذه الرحلة.

إلى من كان حاضرًا في لحظات التعب قبل الفرح، وفي أيام الضغط قبل لحظات النجاح. شكرًا لكل موقف، ولكل كلمة، ولكل لحظة جعلت الطريق أسهل.

إلى **الدكتور أكرم مسوح،**

مشروف هذا المشروع ، لك مني خالص الامتنان على ما قدمته من علم وتوجيه وملاحظات كان لها أثر في الوصول بهذا العمل إلى صورته النهائية.

ثم إلى **نفسي**...

إلى تلك التي تعبت ولم تتوقف،

إلى التي شكّت أحيانًا، لكنها أكملت،

إلى كل مرة شعرت فيها أن الطريق أطول من قدرتها، ثم اختارت أن تستمر.

أهدي هذا الإنجاز لنفسي أيضًا؛

لأنني أعرف كم كان الوصول إليه يستحق.

الحمد لله الذي أتمّ نعمته، ويسّر ما استعصى، وبلغني هذه اللحظة التي طالما حلمت بها ، الحمدلله أولًا وآخرًا،

على طريقٍ لم يكن سهلًا، لكنه كان يستحق أن يُعاش،

وعلى حلمٍ أصبح اليوم حقيقة.

المهندسة ساره اياد عطا

Abstract

The rapid growth of digital technologies has significantly transformed the educational environment, creating a demand for secure and intelligent academic platforms that facilitate communication, collaboration, and knowledge sharing among university communities. Traditional communication methods often lack integration, security, and dedicated academic features, making it difficult for students and faculty members to interact efficiently within a unified environment.

This project presents **UniSphere**, a secure academic community platform specifically designed for university students and professors. The platform provides a centralized environment that enables users to create and share academic posts, communicate through real-time messaging, discover other users, manage connections, receive instant notifications, and participate in a collaborative academic community. To ensure secure authentication and identity management, the system integrates **Clerk Authentication**, supporting both email/password authentication and Google Single Sign-On (SSO). In addition, the platform implements a faculty selection process during the first login and an account verification workflow before granting full system access.

The system was developed using the **MERN Stack**, consisting of **MongoDB**, **Express.js**, **React.js**, and **Node.js**, following a modern client-server architecture. Redux Toolkit was used for state management, while Clerk handled authentication and session management. The platform also incorporates responsive user interface design, secure RESTful APIs, real-time communication, and role-based access control to provide a reliable and scalable solution.

The implementation demonstrates that UniSphere successfully satisfies the functional and non-functional requirements of an academic social networking platform. The developed system offers a secure authentication mechanism, an intuitive user experience, efficient real-time communication, and scalable architecture suitable for future enhancements. The project contributes a practical solution that strengthens academic collaboration while maintaining security, usability, and maintainability.

Table of Contents

List Of Figures

List Of Tables

List of Terms and Abbreviations

Abbreviation	Full Term	Description
AI	Artificial Intelligence	Field of computer science that enables machines to perform intelligent tasks.
API	Application Programming Interface	Interface used for communication between the frontend and backend services.
JWT	JSON Web Token	Secure token used for user authentication and authorization.
MERN	MongoDB, Express.js, React.js, Node.js	Full-stack JavaScript technology stack used to develop UniSphere.
UI	User Interface	The graphical interface through which users interact with the system.
UX	User Experience	Overall experience and usability of the platform.
SPA	Single Page Application	Web application that dynamically updates a single HTML page without full page reloads.
REST	Representational State Transfer	Architectural style used to build backend APIs.

Abbreviation	Full Term	Description
CRUD	Create, Read, Update, Delete	Basic operations performed on system data.
SSO	Single Sign-On	Authentication method allowing users to sign in using their Google account.
OAuth 2.0	Open Authorization 2.0	Authorization protocol used for Google authentication.
Clerk	Clerk Authentication Platform	Authentication and user management service used in UniSphere.
Redux	Redux Toolkit	State management library used in the React application.
MongoDB	MongoDB Database	NoSQL database used to store application data.
Express.js	Express JavaScript Framework	Backend framework used to build REST APIs.
React.js	React JavaScript Library	Frontend library used to build the user interface.
Node.js	Node.js Runtime Environment	JavaScript runtime used for server-side development.
CSS	Cascading Style Sheets	Language used to style web pages.
HTML	HyperText Markup Language	Standard markup language for creating web pages.
JavaScript	JavaScript Programming Language	Main programming language used in both frontend and backend development.

Chapter One Introduction

1.1 Introduction

The rapid development of information and communication technologies has significantly transformed the educational environment, making digital platforms an essential part of modern universities. Academic institutions increasingly depend on online systems to facilitate communication, collaboration, knowledge sharing, and the management of academic activities. These platforms help students, professors, and administrators interact more efficiently while providing quick access to educational resources and services.

Despite these technological advancements, many universities still rely on separate applications for announcements, communication, academic discussions, and resource sharing. This fragmentation often leads to inefficient communication, information loss, and difficulties in collaboration among members of the academic community. Consequently, there is a growing need for an integrated platform that combines these services within a single secure and user-friendly environment.

This research presents **UniSphere**, an Academic Community Platform designed to strengthen communication and collaboration within the university environment. The platform provides a centralized digital space where students and professors can share academic knowledge, exchange information, communicate through real-time messaging, and receive important notifications. By integrating modern web technologies and secure authentication mechanisms, UniSphere aims to improve the overall academic experience while supporting digital transformation in higher education.

1.2 Scientific Problem of the Research

Although universities have adopted various digital technologies to support academic activities, many existing systems remain fragmented and limited in functionality. Students and faculty members often rely on multiple independent platforms for communication, announcements, resource sharing, and academic collaboration. This separation leads to difficulties in accessing information, weak interaction among users, duplicated efforts, and an overall inefficient academic communication process.

In addition, many available platforms are not specifically designed to meet the needs of academic communities. They often lack essential features such as secure identity verification, real-time communication, centralized academic discussions, faculty-based organization, and integrated notification services. These limitations reduce collaboration opportunities and negatively affect the exchange of academic knowledge within the university environment.

Therefore, the scientific problem addressed in this research is the absence of a unified, secure, and intelligent academic community platform that effectively integrates communication, collaboration, user authentication, and knowledge sharing into a single system. Addressing this problem requires the development of a modern platform that enhances interaction among students, professors, and administrators while providing a secure, scalable, and user-friendly environment.

1.3 Research Objectives

The primary objective of this research is to design and develop **UniSphere**, a secure and integrated academic community platform that enhances communication, collaboration, and knowledge sharing among students, professors, and university administrators.

The specific objectives of the research are:

- **Main Objective:**
    - Develop a secure, scalable, and user-friendly academic community platform for higher education institutions.
- **Sub-Objectives:**
    - Implement a secure authentication and authorization mechanism using Clerk Authentication and Google Single Sign-On (SSO).
    - Develop a responsive and intuitive user interface that provides an excellent user experience across different devices.
    - Enable users to create, edit, delete, and interact with academic posts.

- Implement real-time messaging to facilitate effective communication between users.
- Develop a notification system that delivers important updates instantly.
- Allow new users to select their faculty during the onboarding process and complete account verification before accessing the platform.
- Implement user profile and connection management features to strengthen academic networking.
- Build secure RESTful APIs using the MERN Stack architecture.
- Ensure an average system response time of less than **3 seconds** under normal operating conditions.
- Achieve a reliable authentication success rate of at least **90%** during system testing.
- Develop a modular and maintainable software architecture that supports future enhancements and scalability.

1.4 Research Contribution

The main contribution of this research is the development of **UniSphere**, an integrated academic community platform that combines secure authentication, academic communication, social networking, and real-time interaction within a single system. Unlike traditional university systems that provide isolated services, UniSphere offers a unified environment where students and professors can communicate, share academic content, build professional connections, and receive real-time notifications through a modern and user-friendly interface.

The project integrates several modern technologies, including the MERN Stack, Clerk Authentication, Google Single Sign-On (SSO), Redux Toolkit, and Server-Sent Events (SSE), to deliver a secure, scalable, and responsive platform. Furthermore, the implementation of faculty selection, account verification, role-based access, and real-time communication enhances the reliability and usability of

the system while supporting the digital transformation of higher education institutions.

1.5 Beneficiaries of the Research

The proposed **UniSphere** platform is expected to benefit several stakeholders within the university environment, including:

- **Students:** By providing a secure platform for academic communication, knowledge sharing, collaboration, real-time messaging, and interaction with professors and classmates.
- **Professors:** By facilitating communication with students, sharing academic announcements and educational content, and encouraging active participation in academic discussions.
- **University Administration:** By offering a centralized platform that simplifies user management, account verification, academic communication, and administrative supervision.
- **Educational Institutions:** By supporting digital transformation initiatives through the implementation of a modern, secure, and scalable academic community platform.
- **Researchers and Software Developers:** By serving as a practical reference for developing secure academic social networking systems using modern web technologies such as the MERN Stack and Clerk Authentication.

1.6 Organization of the Thesis

This thesis is organized into five chapters, each covering a specific aspect of the research and system development process:

- **Chapter One** introduces the research by presenting the background, scientific problem, research objectives, research contribution, beneficiaries, and the overall organization of the thesis.

- **Chapter Two** reviews the related literature and previous studies relevant to academic community platforms, web-based systems, authentication technologies, and modern software development approaches.
- **Chapter Three** presents the system analysis and design, including the functional and non-functional requirements, use case diagrams, class diagrams, sequence diagrams, database design, system architecture, and other UML models used during the development process.
- **Chapter Four** describes the implementation and testing of the proposed system. It explains the development tools, technologies, implementation details, user interface, system modules, testing procedures, and the evaluation of the system's functionality and performance.
- **Chapter Five** concludes the research by summarizing the achievements of the project, discussing its limitations, and presenting recommendations and future work for further enhancement of the UniSphere platform.

Chapter Two Analytical Study

2.1 Introduction

This chapter presents the analytical study of the proposed **UniSphere** platform. It focuses on identifying the system requirements, defining the functional and non-functional specifications, and describing the development process adopted throughout the project. The chapter also introduces the project schedule, the software architectures used in the implementation, and the development methodology followed during the system construction.

A comprehensive analysis of the system requirements was conducted to ensure that the developed platform satisfies both user expectations and technical constraints. The identified requirements served as the foundation for the design and implementation phases, ensuring that every feature contributes to the overall objectives of the project.

Furthermore, this chapter discusses the architectural design adopted in the project, including the **Client–Server Architecture** and the **Model–View–Controller (MVC)** architectural pattern, explaining their roles and integration within the system. Finally, the chapter describes the **Scrum** development methodology used to manage the project through iterative development, continuous testing, and incremental delivery of system features.

2.2 Functional Requirements

Functional requirements describe the services, operations, and behaviors that the proposed system must perform to satisfy user needs and achieve the project objectives. These requirements define how the system responds to user actions and specify the core functionalities available to different user roles. The functional requirements of **UniSphere** were identified through system analysis and user requirements gathering, ensuring that the platform supports secure authentication, academic communication, content sharing, and user management.

Table 1-Functional Requirements of the Proposed UniSphere System

ID	Requirement Name	Description	Type
ID-01	User Registration	Allow users to create a new account using Clerk Authentication.	Functional
ID-02	User Login	Allow users to log in using Email/Password or Google SSO.	Functional
ID-03	User Logout	Allow users to securely log out of the system.	Functional
ID-04	Student Verification	Verify student accounts before granting full access.	Functional
ID-05	Professor Verification	Verify professor accounts before granting full access.	Functional
ID-06	Faculty Selection	Allow first-time users to select their faculty.	Functional
ID-07	View User Profile	Allow users to view their profile information.	Functional
ID-08	Edit User Profile	Allow users to update their profile details and photo.	Functional
ID-09	Create Post	Allow users to publish academic posts.	Functional
ID-10	Edit Post	Allow users to edit their own posts.	Functional
ID-11	Delete Post	Allow users to remove their own posts.	Functional
ID-12	View Posts	Display posts on the news feed.	Functional
ID-13	Like Post	Allow users to like and unlike posts.	Functional
ID-14	Add Comment	Allow users to comment on posts.	Functional

ID	Requirement Name	Description	Type
ID-15	Edit Comment	Allow users to edit their comments.	Functional
ID-16	Delete Comment	Allow users to delete their comments.	Functional
ID-17	Search Users	Allow users to search for students and professors.	Functional
ID-18	Discover Users	Display suggested users to connect with.	Functional
ID-19	Send Connection Request	Allow users to send connection requests.	Functional
ID-20	Accept / Reject Connection	Allow users to accept or reject requests.	Functional
ID-21	Real-Time Messaging	Enable private real-time messaging between users.	Functional
ID-22	Notifications	Notify users of messages, likes, comments, and requests.	Functional
ID-23	Report Post	Allow users to report inappropriate posts.	Functional
ID-24	Manage Reports	Allow administrators to review reported posts.	Functional
ID-25	View Notifications	Allow users to view notification history.	Functional
ID-26	Role-Based Access Control	Restrict system features based on user roles.	Functional
ID-27	Responsive User Interface	Provide an adaptive interface across different devices.	Functional
ID-28	Session Management	Maintain secure user sessions after authentication.	Functional

ID	Requirement Name	Description	Type
ID-29	Account Approval Workflow	Prevent access until account verification is completed.	Functional
ID-30	Feed Management	Display personalized academic content based on user activity.	Functional

- Each functional requirement was implemented, tested, and traced throughout the software development life cycle to ensure complete coverage from system analysis to implementation and testing. The traceability between requirements, implementation files, and test cases ensures that every system function has been successfully developed and validated.

2.3 Non-Functional Requirements

Non-functional requirements define the quality attributes and operational characteristics of the proposed system rather than its specific functionalities. These requirements ensure that **UniSphere** is reliable, secure, efficient, maintainable, and capable of providing a satisfactory user experience. They describe the constraints and performance standards that the system must satisfy to operate effectively in a university environment.

Table 2-Non-Functional Requirements of the Proposed UniSphere System

Requirement ID	Requirement Name	Description	Type
NFR-01	Performance	The system shall respond to user requests within **3 seconds** under normal operating conditions.	Non-Functional
NFR-02	Security	The system shall protect user data using secure	Non-Functional

Requirement ID	Requirement Name	Description	Type
		authentication, encrypted communication (HTTPS), and Clerk Authentication.	
NFR-03	Availability	The platform shall be available to users at any time with minimal downtime.	Non-Functional
NFR-04	Reliability	The system shall provide consistent and error-free operation during normal usage.	Non-Functional
NFR-05	Scalability	The system shall support an increasing number of users without significant degradation in performance.	Non-Functional
NFR-06	Usability	The platform shall provide an intuitive, user-friendly, and responsive interface.	Non-Functional
NFR-07	Maintainability	The software architecture shall support easy maintenance, debugging, and future enhancements.	Non-Functional
NFR-08	Compatibility	The system shall operate correctly on modern web browsers and different screen sizes.	Non-Functional
NFR-09	Data Integrity	The system shall ensure the accuracy and consistency of stored information.	Non-Functional

2.4 Project Schedule (Gantt Chart)

A project schedule was developed to organize and monitor the implementation of the **UniSphere** platform throughout the development lifecycle. The schedule outlines the major phases of the project, including requirement analysis, system design, implementation, testing, documentation, and final deployment. Using a Gantt Chart allowed the development team to allocate tasks efficiently, monitor project progress, and ensure that all milestones were completed within the planned timeframe.

The Gantt Chart also facilitated effective time management by illustrating the sequence, duration, and dependencies of project activities. This approach helped maintain a structured development process and ensured that each phase was completed before moving to the next stage of the project.

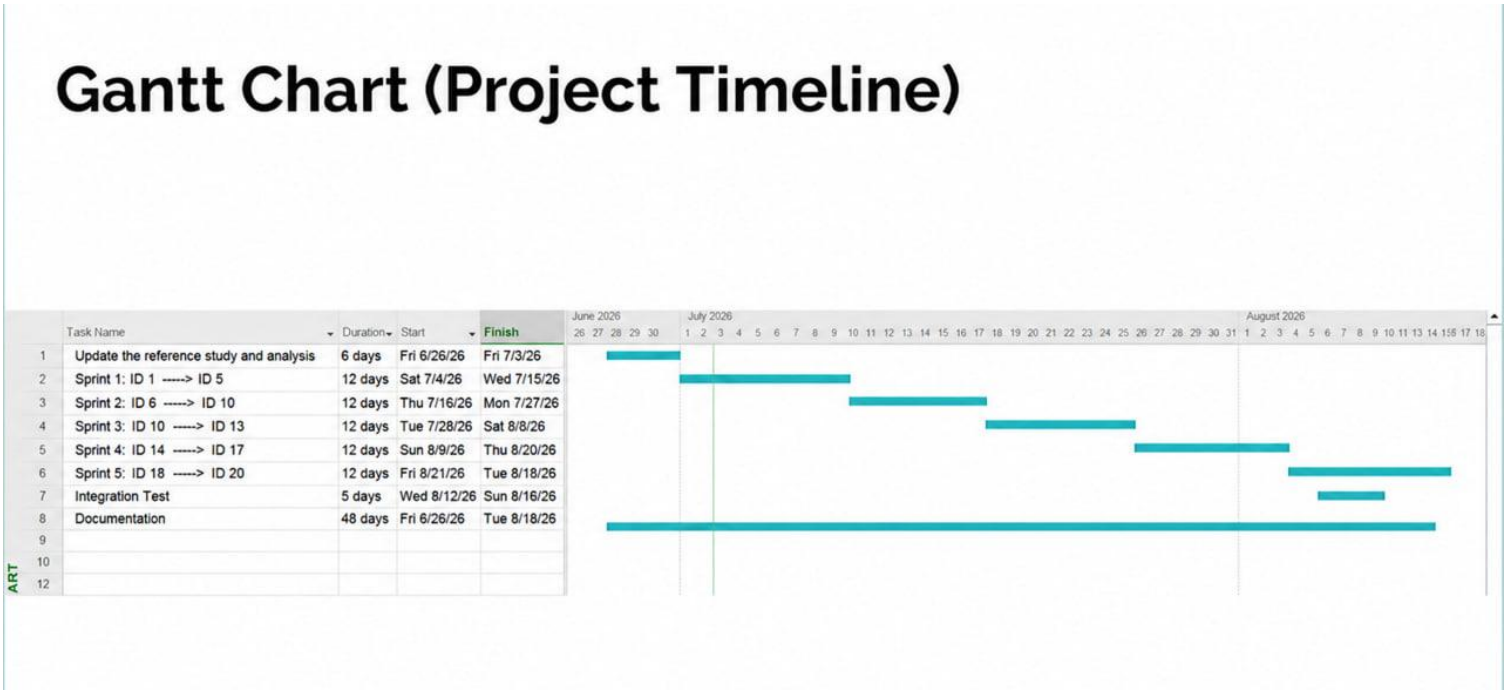

Figure 1- Project Gantt Chart

2.5 System Architecture

The architecture of a software system defines the overall structure of the application and describes how its different components interact to provide the

required functionality. Selecting an appropriate software architecture is essential for ensuring scalability, maintainability, performance, and ease of future development.

The proposed **UniSphere** platform adopts two complementary architectural approaches: the **Client–Server Architecture** and the **Model–View–Controller (MVC) Architecture**. The Client–Server Architecture manages the communication between the frontend and backend, while the MVC pattern organizes the backend components into separate layers responsible for handling business logic, user requests, and data management. The combination of these architectures provides a modular, scalable, and maintainable system suitable for modern web applications.

2.5.1 Client–Server Architecture

The Client–Server Architecture is a distributed software architecture in which the application is divided into two main components: the **client**, responsible for presenting the user interface and handling user interactions, and the **server**, responsible for processing requests, executing business logic, communicating with the database, and returning the appropriate responses.

In the **UniSphere** platform, the client side was developed using **React.js** and **Tailwind CSS**, providing users with a responsive and interactive interface. The server side was implemented using **Node.js** and **Express.js**, exposing RESTful APIs that process requests, enforce business rules, communicate with the MongoDB database, and return JSON responses to the client. Communication between both components takes place over the HTTP/HTTPS protocol.

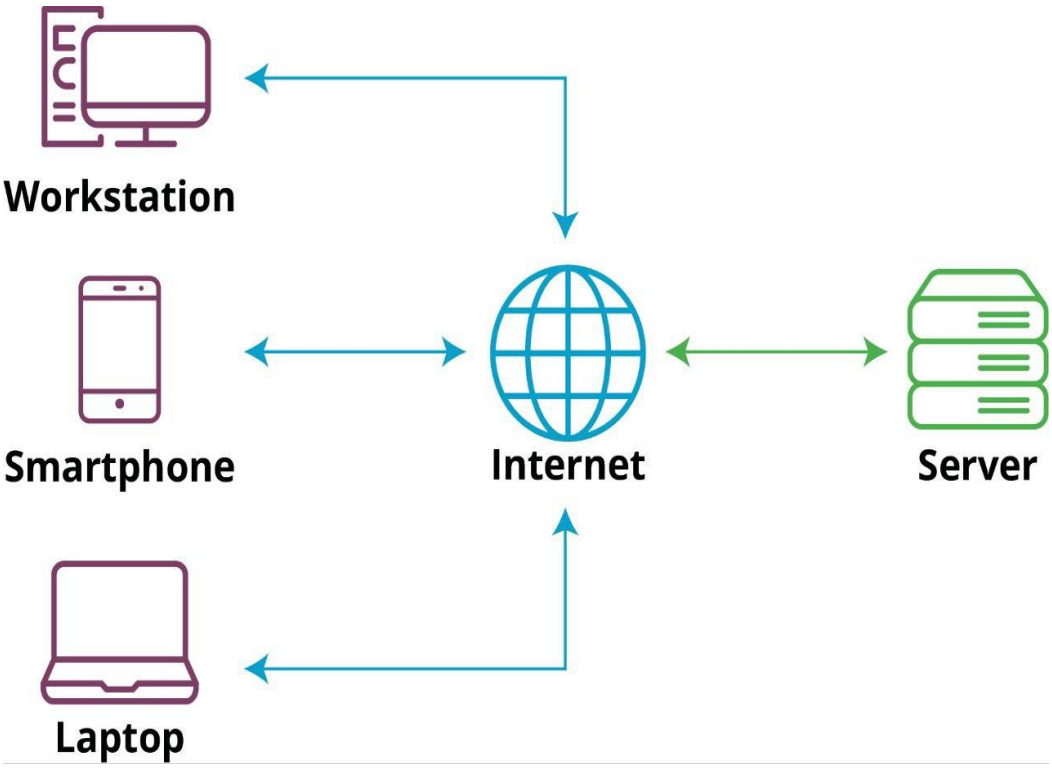

Figure 2- Client–Server Architecture of the UniSphere System

Advantages of Client–Server Architecture

- Separation between the presentation layer and business logic.
- Easier maintenance and future system enhancements.
- Supports scalability by allowing independent expansion of server resources.
- Improves security by centralizing data processing on the server.
- Facilitates communication between multiple clients and a single backend service.
- Simplifies database management through centralized access.

Disadvantages of Client–Server Architecture

- The system depends heavily on server availability.
- High server workload may affect system performance under heavy traffic.

- Network failures may interrupt communication between the client and server.
- Requires secure communication mechanisms to protect transmitted data.

2.5.2 Model–View–Controller (MVC) Architecture

The **Model–View–Controller (MVC)** architecture is a widely adopted software design pattern that separates an application into three independent components: the **Model**, the **View**, and the **Controller**. This separation improves code organization, enhances maintainability, and simplifies the development process by assigning a specific responsibility to each component.

In the UniSphere platform, the MVC architecture is implemented on the backend. The **Model** represents the application's data and interacts with the MongoDB database using Mongoose schemas. The **Controller** processes incoming requests, executes the business logic, validates user input, and communicates with the models. The **View** is represented by the JSON responses sent through the RESTful APIs, which are consumed by the React frontend to present information to users.

Components of the MVC Architecture

Model

The Model manages the application's data, database operations, and business entities. It is responsible for creating, retrieving, updating, and deleting data stored in MongoDB.

View

The View represents the output presented to the user. In the UniSphere project, the frontend developed with **React.js** acts as the presentation layer by displaying the JSON data received from the backend APIs.

Controller

The Controller serves as the intermediary between the client requests and the data layer. It receives HTTP requests, validates the input, executes the appropriate

business logic, communicates with the Model, and returns the appropriate response to the client.

Advantages of MVC Architecture

- Clear separation of responsibilities between system components.
- Improves code readability and maintainability.
- Simplifies debugging and software testing.
- Supports parallel development by allowing frontend and backend developers to work independently.
- Enhances scalability through modular application design.
- Facilitates future feature additions without affecting existing modules.

Disadvantages of MVC Architecture

- Requires a well-organized project structure.
- Introduces additional files and folders compared to simpler architectures.
- May increase development complexity for small applications.
- Requires developers to understand the interaction between the three components.

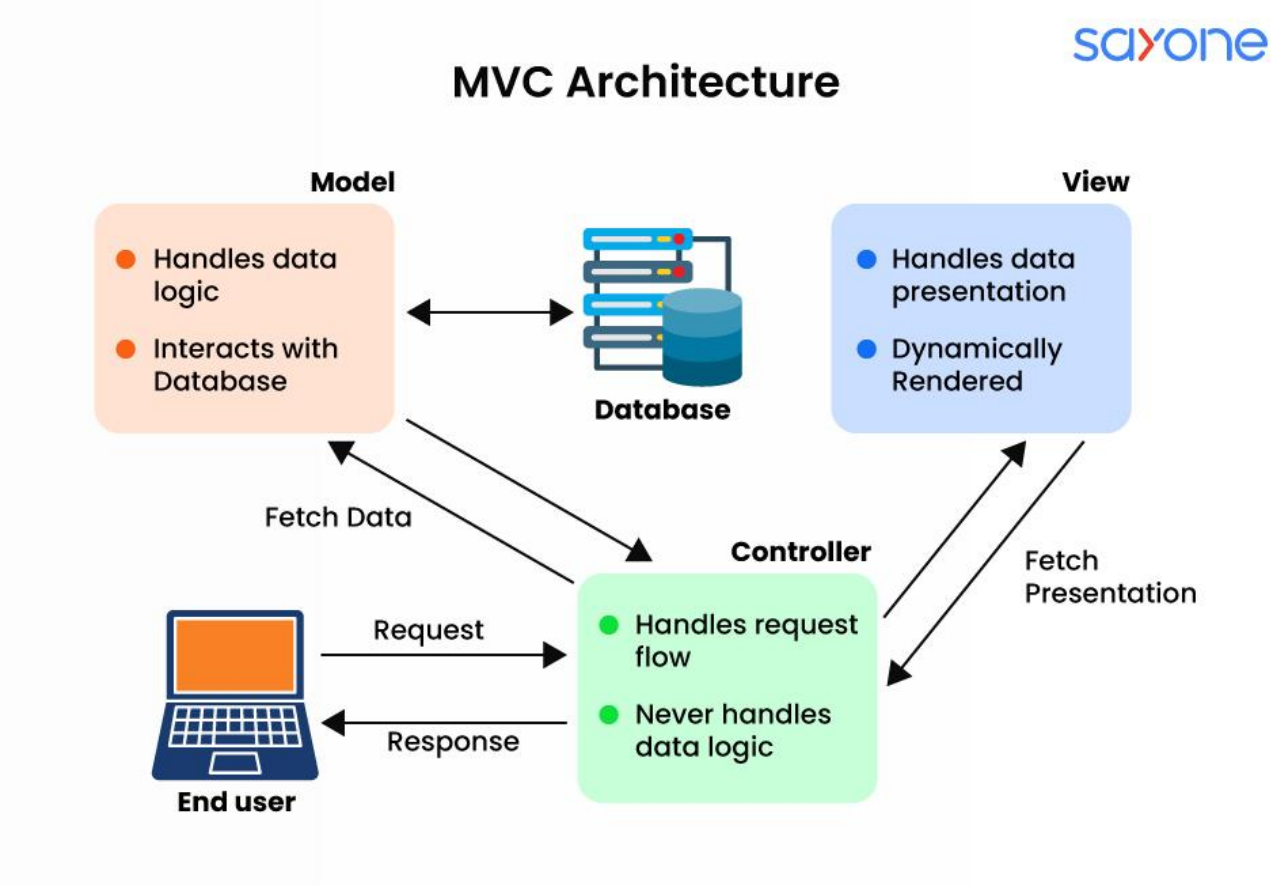

Figure 3- MVC Architecture of the UniSphere System

2.5.3 Integration of Client–Server and MVC Architectures in UniSphere

The UniSphere platform combines the **Client–Server Architecture** with the **Model–View–Controller (MVC)** architectural pattern to achieve a modular, scalable, and maintainable system. While the Client–Server Architecture defines the communication between the frontend and the backend, the MVC architecture organizes the internal structure of the backend application.

On the client side, the user interacts with the system through a responsive web interface developed using **React.js** and **Tailwind CSS**. User actions, such as authentication, creating posts, sending messages, updating profiles, and managing connections, generate HTTP requests that are sent to the backend through RESTful APIs.

Once a request reaches the server, the **Express.js** application routes it to the appropriate controller. The controller validates the received data, executes the required business logic, and communicates with the corresponding model. The model then performs the necessary database operations using **MongoDB** through **Mongoose**. After completing the requested operation, the server returns a JSON response to the client, where React dynamically updates the user interface without requiring a full page reload.

This integration allows each architectural layer to perform its specific responsibility independently. The Client–Server Architecture manages communication between users and the server, while the MVC pattern ensures a clear separation between data management, business logic, and request handling. As a result, the UniSphere platform becomes easier to maintain, more secure, highly scalable, and capable of supporting future enhancements with minimal impact on the existing system.

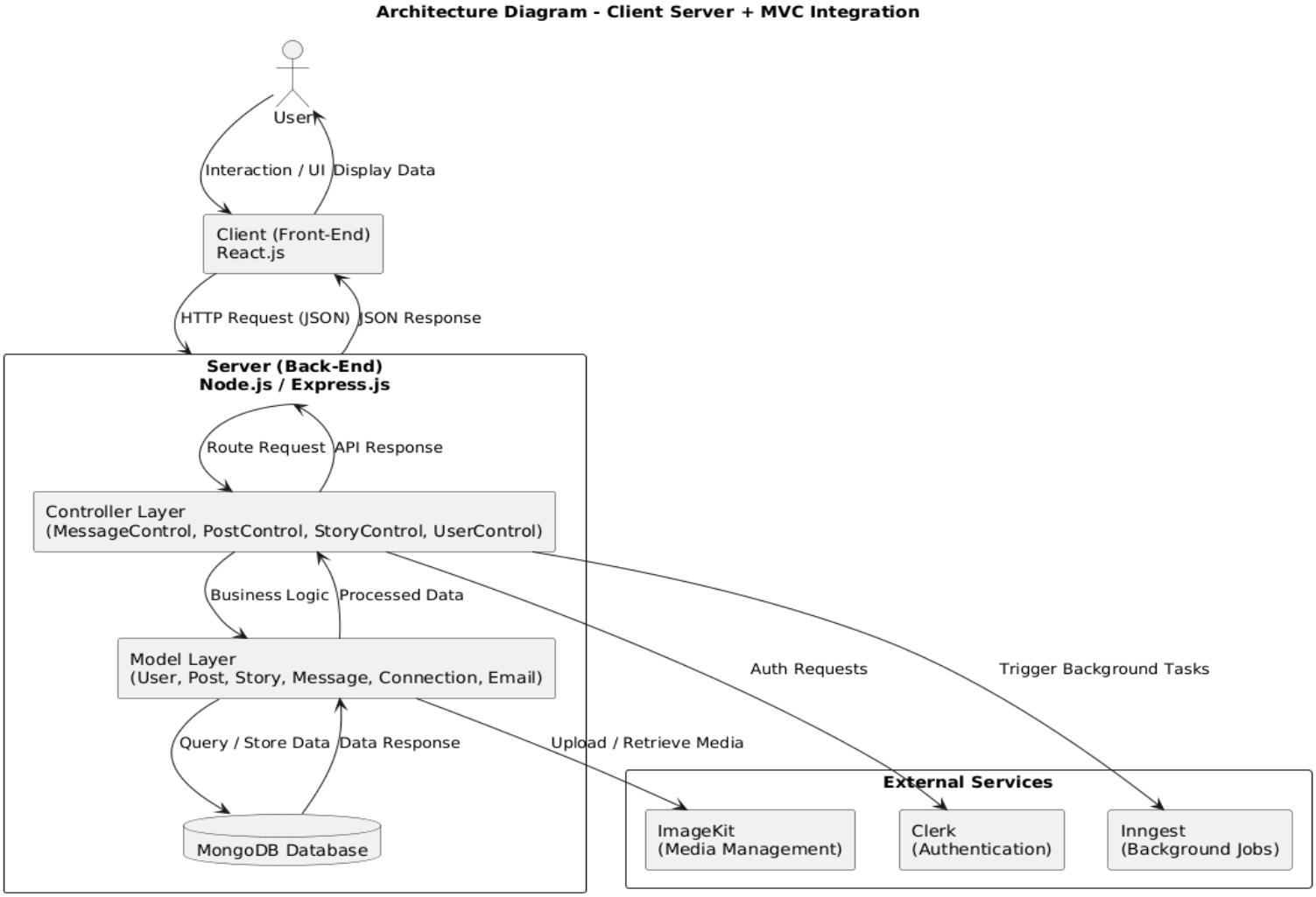

Figure 4- Integration Between Client–Server and MVC Architectures in UniSphere

2.6 Development Methodology (Scrum Methodology)

The development of the **UniSphere** platform followed the **Scrum** methodology, one of the most widely used Agile software development frameworks. Scrum was selected because it supports iterative development, continuous feedback, effective team collaboration, and incremental delivery of software features. This methodology enabled the development team to divide the project into manageable phases, monitor progress regularly, and respond efficiently to changing requirements throughout the development process.

The project was organized into a series of **Sprints**, where each sprint focused on implementing a specific group of features. During every sprint, the development team analyzed the assigned tasks, implemented the required functionalities, performed testing, and reviewed the completed work before proceeding to the next iteration. This iterative approach improved software quality and reduced development risks.

Scrum Roles

The Scrum methodology defines three primary roles that contributed to the successful development of the UniSphere platform.

Product Owner

The Product Owner was responsible for defining the project requirements, prioritizing the product backlog, and ensuring that the developed functionalities satisfied the project objectives and user needs.

Scrum Master

The Scrum Master coordinated the development process, facilitated communication among team members, ensured that Scrum practices were followed correctly, and removed obstacles that could affect project progress.

Development Team

The Development Team was responsible for designing, implementing, testing, and integrating the different components of the UniSphere platform, including the frontend, backend, database, authentication system, and user interface.

Scrum Workflow in UniSphere

The development process followed these iterative stages:

1. Requirement Analysis
2. Product Backlog Preparation
3. Sprint Planning

4. System Design
5. Implementation
6. Testing and Bug Fixing
7. Sprint Review
8. Sprint Retrospective
9. Deployment

Advantages of Using Scrum

- Supports incremental software development.
- Enables continuous testing and quality improvement.
- Facilitates effective communication among team members.
- Allows rapid adaptation to changing project requirements.
- Reduces development risks through frequent reviews.
- Improves project planning and task management.
- Ensures continuous delivery of functional software.

Challenges of Scrum

- Requires continuous collaboration among team members.
- Depends on effective communication and regular meetings.
- Project scope changes may increase development effort.
- Requires disciplined sprint planning and task management.

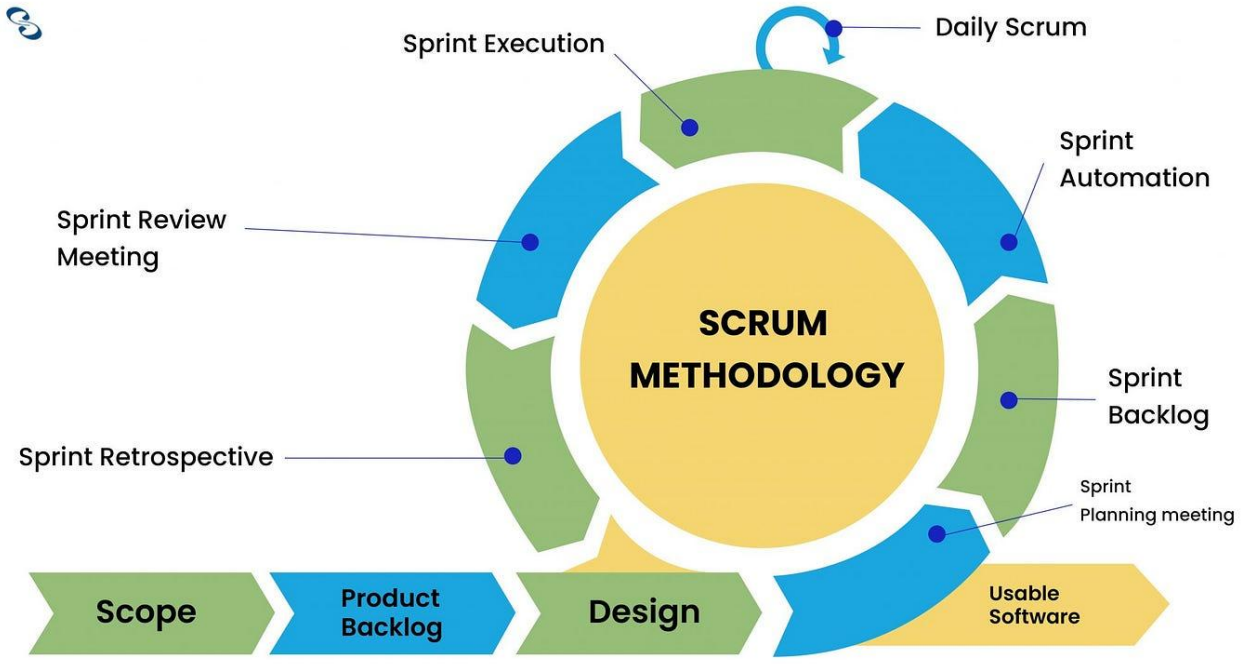

Figure 5- Scrum Development Process

2.7 Chapter Summary

This chapter presented the analytical study of the proposed UniSphere platform. It introduced the functional and non-functional requirements that define the system's behavior and quality attributes. Furthermore, it described the project schedule using a Gantt Chart, explained the software architectures adopted in the implementation, including the Client–Server and MVC architectures, and discussed their integration within the proposed system. Finally, the Scrum methodology used during the software development process was presented, demonstrating how iterative development and continuous evaluation contributed to the successful implementation of the platform.

Chapter Three – System Design

3.1 Introduction

This chapter presents the design phase of the proposed **UniSphere** platform. System design plays a vital role in transforming the functional and non-functional requirements identified during the analysis phase into a structured and implementable software solution. It provides a clear representation of the system's architecture, user interactions, data organization, and software components before the implementation stage.

The design of UniSphere was developed using standard **Unified Modeling Language (UML)** diagrams to illustrate different aspects of the system. These diagrams include the **Use Case Diagram**, **Sequence Diagrams**, **Activity Diagrams**, and the **Class Diagram**, each providing a different perspective of the system's behavior and structure. In addition, an **Entity Relationship Diagram (ERD)** was created to represent the data model implemented using **MongoDB**, reflecting the relationships among the main entities of the platform.

Furthermore, this chapter presents the **Use Case Specifications**, **Test Cases**, **Requirements Traceability Matrix (RTM)**, and the **Site Map**, which collectively describe the interaction between users and the system, validate the implemented functionalities, and demonstrate the traceability between requirements, implementation, and testing. These design artifacts ensure that the developed platform is well-structured, maintainable, and capable of supporting future enhancements.

3.2 Use Case Diagram

A **Use Case Diagram** is one of the most important UML diagrams used during the system design phase. It provides a high-level view of the system by illustrating the interactions between external users (actors) and the functionalities provided by the system. The diagram helps identify the services available to each actor and defines the overall scope of the proposed system.

In the UniSphere platform, the Use Case Diagram represents the interaction between the three primary actors: **Student**, **Professor**, and **Administrator**. Each actor performs a specific set of operations according to their assigned permissions. Students and professors can authenticate, manage their profiles, create academic

posts, exchange messages, manage connections, receive notifications, and interact with other users. The administrator is responsible for supervising the platform by reviewing reported content and maintaining the integrity of the system.

The Use Case Diagram was designed based on the functional requirements identified during the analysis phase and serves as the foundation for developing the Sequence Diagrams, Activity Diagrams, Use Case Specifications, Test Cases, and Requirements Traceability Matrix presented in the following sections.

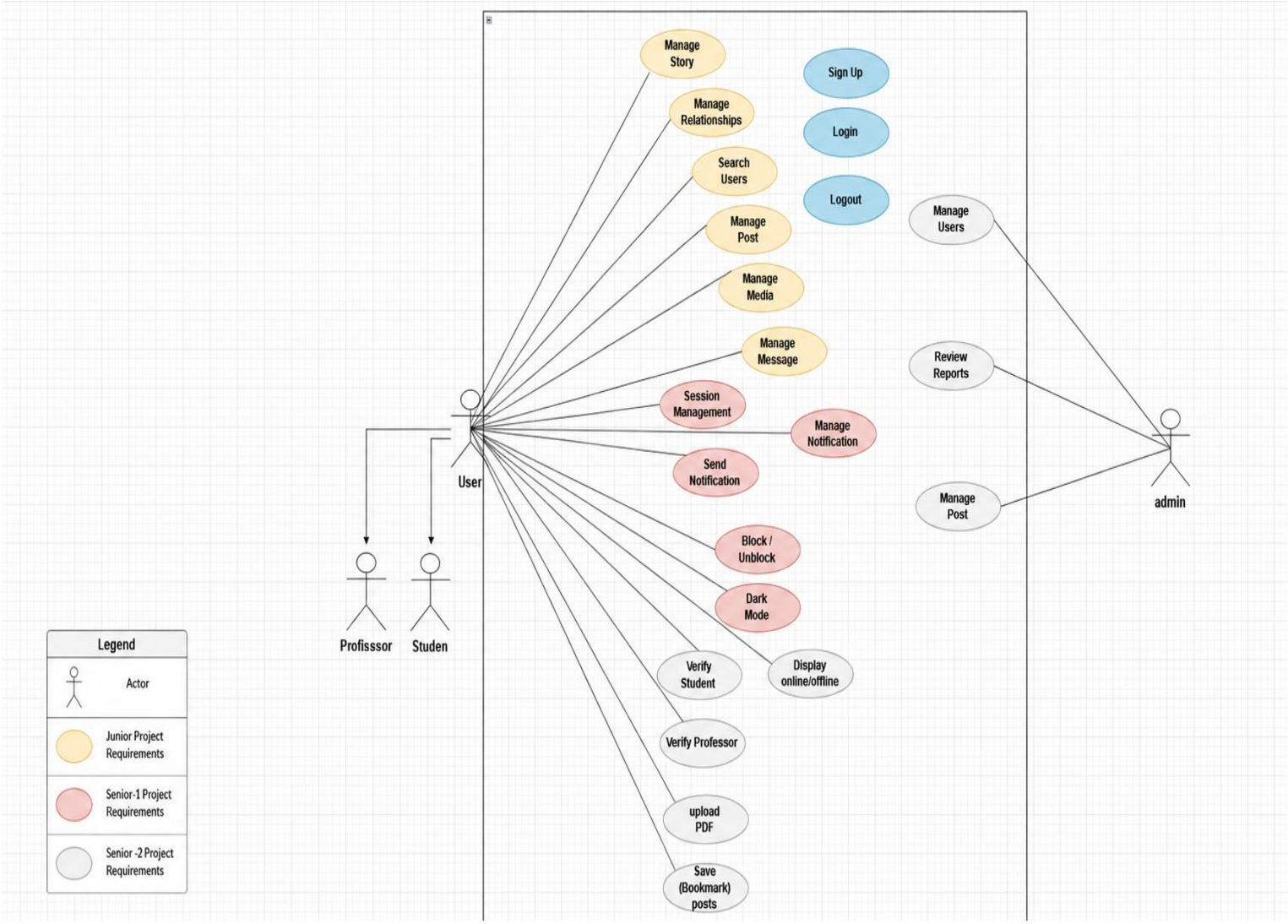

Figure 6- Use Case Diagram of the UniSphere System

3.3 Use Case Specifications

Use Case Specifications provide a detailed description of each system functionality identified in the Use Case Diagram. Each specification defines the purpose of the use case, the participating actor(s), the preconditions, the normal execution flow, alternative flows, postconditions, and any exceptions that may occur during execution. These specifications serve as a bridge between the system requirements and the implementation phase, ensuring that every functional requirement is clearly defined and correctly implemented.

Use Case Specification – UC-01

Table 3.1: Use Case Specification – User Login

Item	**Description**
Use Case ID	UC-01
Use Case Name	User Login
Primary Actor	Student, Professor
Description	Allows registered users to securely authenticate using Email/Password or Google Single Sign-On (SSO).
Preconditions	The user must have a registered account.
Trigger	The user opens the login page and submits valid credentials.
Main Flow	1. The user enters email and password or selects Google Sign-In.2. The system validates the credentials.3. The system creates a secure session.4. The user is redirected to the appropriate page based on the account status.
Alternative Flow	If the credentials are invalid, the system displays an authentication error message.

Item	Description
Postconditions	The user is successfully authenticated and an active session is created.

Table 3- Use Case Specification – User Login

Table 3.2: Use Case Specification – Student Verification

Item	Description
Use Case ID	UC-02
Use Case Name	Student Verification
Primary Actor	Student
Description	Verifies the student's university identity before granting full access to the platform.
Preconditions	The student has successfully authenticated.
Trigger	The student submits the required verification information.
Main Flow	1. The student uploads the required verification data.2. The system validates the submitted information.3. The verification request is processed.4. The account status is updated.
Alternative Flow	Invalid or incomplete information results in a pending or rejected verification request.
Postconditions	The student's account is verified or remains pending review.

Table 4- Use Case Specification – Student Verification

Table 3.3: Use Case Specification – Professor Verification

Item	Description
Use Case ID	UC-03
Use Case Name	Professor Verification
Primary Actor	Professor
Description	Verifies the professor's identity before enabling professor privileges within the platform.
Preconditions	The professor has successfully authenticated.
Trigger	The professor submits the required verification information.
Main Flow	1. The professor uploads the required documents.2. The system validates the information.3. The verification process is completed.4. Professor privileges are activated.
Alternative Flow	If verification fails, the account remains in the pending verification state.
Postconditions	The professor account is verified or remains pending approval.

Table 5- Use Case Specification – Professor Verification

Table 3.4: Use Case Specification – Faculty Selection

Item	Description
Use Case ID	UC-04
Use Case Name	Faculty Selection
Primary Actor	Student, Professor

Item	Description
Description	Allows first-time users to select their faculty during the onboarding process.
Preconditions	The user has successfully authenticated for the first time.
Trigger	The onboarding process begins.
Main Flow	1. The system displays the available faculties.2. The user selects a faculty.3. The selected faculty is stored.4. The onboarding process continues.
Alternative Flow	If no faculty is selected, the user cannot continue.
Postconditions	The selected faculty is successfully saved in the user's profile.

Table 6- Use Case Specification – Faculty Selection

Table 3.5: Use Case Specification – Profile Management

Item	Description
Use Case ID	UC-05
Use Case Name	Profile Management
Primary Actor	Student, Professor
Description	Allows users to view and update their profile information, profile picture, and personal details.
Preconditions	The user is authenticated.
Trigger	The user opens the profile page.

Item	Description
Main Flow	1. The user views the profile.2. The user edits profile information.3. The system validates the entered data.4. The updated information is saved successfully.
Alternative Flow	Invalid or incomplete information is rejected and an error message is displayed.
Postconditions	The user's profile is successfully updated.

Table 7- Use Case Specification – Profile Management

Table 3.6: Use Case Specification – Create Post

Item	Description
Use Case ID	UC-06
Use Case Name	Create Post
Primary Actor	Student, Professor
Description	Allows users to create and publish academic posts.
Preconditions	The user is authenticated.
Trigger	The user selects "Create Post".
Main Flow	1. The user enters the post content.2. The user optionally uploads an image.3. The user publishes the post.4. The system stores and displays the post.
Alternative Flow	Empty posts cannot be published.
Postconditions	The post appears in the news feed.

Table 8- Use Case Specification – Create Post

Table 3.7: Use Case Specification – Comment Management

Item	Description
Use Case ID	UC-07
Use Case Name	Comment Management
Primary Actor	Student, Professor
Description	Allows users to add, edit, and delete comments on posts.
Preconditions	The user is authenticated and the selected post exists.
Trigger	The user selects the comment option.
Main Flow	1. The user writes a comment.2. The system validates the comment.3. The comment is stored.4. The comment is displayed below the post.
Alternative Flow	Empty comments cannot be submitted.
Postconditions	The comment is successfully added, updated, or removed.

Table 9- Use Case Specification – Comment Management

Table 3.8: Use Case Specification – Like Post

Item	Description
Use Case ID	UC-08
Use Case Name	Like Post
Primary Actor	Student, Professor

Item	Description
Description	Allows users to like or unlike academic posts.
Preconditions	The user is authenticated.
Trigger	The user clicks the Like button.
Main Flow	1. The user clicks Like.2. The system updates the like count.3. The updated result is displayed immediately.
Alternative Flow	Clicking Like again removes the previous like.
Postconditions	The post interaction is updated successfully.

Table 10- Use Case Specification – Like Post

Table 3.9: Use Case Specification – Connection Management

Item	Description
Use Case ID	UC-09
Use Case Name	Connection Management
Primary Actor	Student, Professor
Description	Allows users to send, accept, reject, and remove connection requests.
Preconditions	Both users have authenticated accounts.
Trigger	The user selects another user's profile.

Item	Description
Main Flow	1. The user sends a connection request.2. The recipient receives the request.3. The recipient accepts or rejects the request.4. The connection list is updated.
Alternative Flow	The recipient rejects or ignores the request.
Postconditions	The connection status is updated successfully.

Table 11- Use Case Specification – Connection Management

Table 3.10: Use Case Specification – Real-Time Messaging

Item	Description
Use Case ID	UC-10
Use Case Name	Real-Time Messaging
Primary Actor	Student, Professor
Description	Allows connected users to exchange private messages in real time.
Preconditions	The users are authenticated and connected.
Trigger	The user opens a conversation.
Main Flow	1. The user writes a message.2. The message is sent to the server.3. The recipient receives the message instantly.4. The conversation is updated in real time.
Alternative Flow	If the recipient is offline, the message is stored and delivered when the recipient reconnects.

Item	Description
Postconditions	The message is successfully stored and displayed to both users.

Table 12- Use Case Specification – Real-Time Messaging

Table 3.11: Use Case Specification – Notification Management

Item	Description
Use Case ID	UC-11
Use Case Name	Notification Management
Primary Actor	Student, Professor
Description	Allows users to receive and manage real-time notifications related to messages, posts, comments, and connection requests.
Preconditions	The user is authenticated.
Trigger	A system event occurs.
Main Flow	1. A new event is generated.2. The system creates a notification.3. The notification is delivered to the user.4. The user views the notification.
Alternative Flow	If the user is offline, the notification is stored until the next login.
Postconditions	The notification is successfully delivered and recorded.

Table 13- Use Case Specification – Notification Management

Table 3.12: Use Case Specification – Search Users

Item	Description
Use Case ID	UC-12
Use Case Name	Search Users
Primary Actor	Student, Professor
Description	Allows users to search for students and professors within the platform.
Preconditions	The user is authenticated.
Trigger	The user enters a search keyword.
Main Flow	1. The user enters a keyword.2. The system searches the database.3. Matching users are displayed.
Alternative Flow	If no users match the search criteria, an empty result message is displayed.
Postconditions	The matching users are displayed successfully.

Table 14- Use Case Specification – Search Users

Table 3.13: Use Case Specification – View Feed

Item	Description
Use Case ID	UC-13
Use Case Name	View Feed
Primary Actor	Student, Professor
Description	Allows users to browse the latest academic posts published by the community.
Preconditions	The user is authenticated.

Item	Description
Trigger	The user opens the Feed page.
Main Flow	1. The system retrieves posts.2. The posts are sorted.3. The feed is displayed.
Alternative Flow	If there are no posts, an empty feed message is displayed.
Postconditions	The news feed is displayed successfully.

Table 15- Use Case Specification – View Feed

Table 3.14: Use Case Specification – View User Profile

Item	Description
Use Case ID	UC-14
Use Case Name	View User Profile
Primary Actor	Student, Professor
Description	Allows users to view their own profile or another user's profile.
Preconditions	The user is authenticated.
Trigger	The user selects a profile.
Main Flow	1. The profile request is sent.2. The system retrieves profile information.3. The profile page is displayed.
Alternative Flow	If the profile does not exist, an error message is displayed.
Postconditions	The requested profile is displayed successfully.

Table 16- Use Case Specification – View User Profile

Table 3.15: Use Case Specification – Report Post

Item	Description
Use Case ID	UC-15
Use Case Name	Report Post
Primary Actor	Student, Professor
Description	Allows users to report inappropriate or harmful posts to the administrator.
Preconditions	The user is authenticated.
Trigger	The user selects "Report Post".
Main Flow	1. The user submits a report.2. The system records the report.3. The administrator receives the report.
Alternative Flow	Duplicate reports may be ignored according to the system rules.
Postconditions	The report is stored successfully.

Table 17- Use Case Specification – Report Post

Table 3.16: Use Case Specification – Review Reports

Item	Description
Use Case ID	UC-16
Use Case Name	Review Reports
Primary Actor	Administrator

Item	Description
Description	Allows administrators to review reported posts and take appropriate actions.
Preconditions	The administrator is authenticated.
Trigger	The administrator opens the Reports page.
Main Flow	1. The system displays all reports.2. The administrator reviews each report.3. An appropriate action is selected.
Alternative Flow	Invalid reports can be dismissed.
Postconditions	The selected report is processed successfully.

Table 18- Use Case Specification – Review Reports

Table 3.17: Use Case Specification – Logout

Item	Description
Use Case ID	UC-17
Use Case Name	Logout
Primary Actor	Student, Professor, Administrator
Description	Allows authenticated users to securely end their current session.
Preconditions	The user is logged in.
Trigger	The user selects the Logout option.
Main Flow	1. The user clicks Logout.2. The current session is terminated.3. The login page is displayed.

Item	Description
Alternative Flow	None.
Postconditions	The session is terminated successfully.

Table 19- Use Case Specification – Logout

Table 3.18: Use Case Specification – View Notifications

Item	Description
Use Case ID	UC-18
Use Case Name	View Notifications
Primary Actor	Student, Professor
Description	Allows users to view all received notifications.
Preconditions	The user is authenticated.
Trigger	The user opens the Notifications page.
Main Flow	1. The system retrieves notification records.2. The notifications are displayed.3. The user reviews them.
Alternative Flow	If there are no notifications, an empty list is displayed.
Postconditions	The notification history is successfully displayed.

Table 20- Use Case Specification – View Notifications

Table 3.19: Use Case Specification – Manage Verification Requests

Item	Description
Use Case ID	UC-19
Use Case Name	Manage Verification Requests
Primary Actor	Administrator
Description	Allows administrators to approve or reject user verification requests.
Preconditions	The administrator is authenticated.
Trigger	A verification request is submitted.
Main Flow	1. The administrator reviews the request.2. The administrator approves or rejects it.3. The account status is updated.
Alternative Flow	The request may remain pending if additional information is required.
Postconditions	The verification status is updated successfully.

Table 21- Use Case Specification – Manage Verification Requests

---

Table 3.20: Use Case Specification – Manage User Sessions

Item	Description
Use Case ID	UC-20
Use Case Name	Manage User Sessions
Primary Actor	System
Description	The system manages user sessions, authentication status, and secure access throughout the application lifecycle.
Preconditions	A user has successfully authenticated.

Item	Description
Trigger	Session creation, validation, or expiration occurs.
Main Flow	1. The system creates a secure session after login.2. The session is validated during user activity.3. The session expires after logout or timeout.
Alternative Flow	Invalid or expired sessions require the user to authenticate again.
Postconditions	Secure session management is maintained throughout system usage.

Table 22- Use Case Specification – Manage User Sessions

3.4 Sequence Diagrams

3.4 Introduction

Sequence diagrams illustrate the chronological interaction between system components during the execution of a specific use case. They describe how objects communicate by exchanging messages in a particular order to accomplish a system function. These diagrams help visualize the dynamic behavior of the system and verify that each functional requirement is correctly implemented.

The following sequence diagrams represent the main operations performed within the UniSphere platform. Each diagram corresponds to one of the use cases presented in the previous section and demonstrates the interaction between the user, the frontend application, the backend server, and the database.

Figure 3.2: Sequence Diagram for User Login

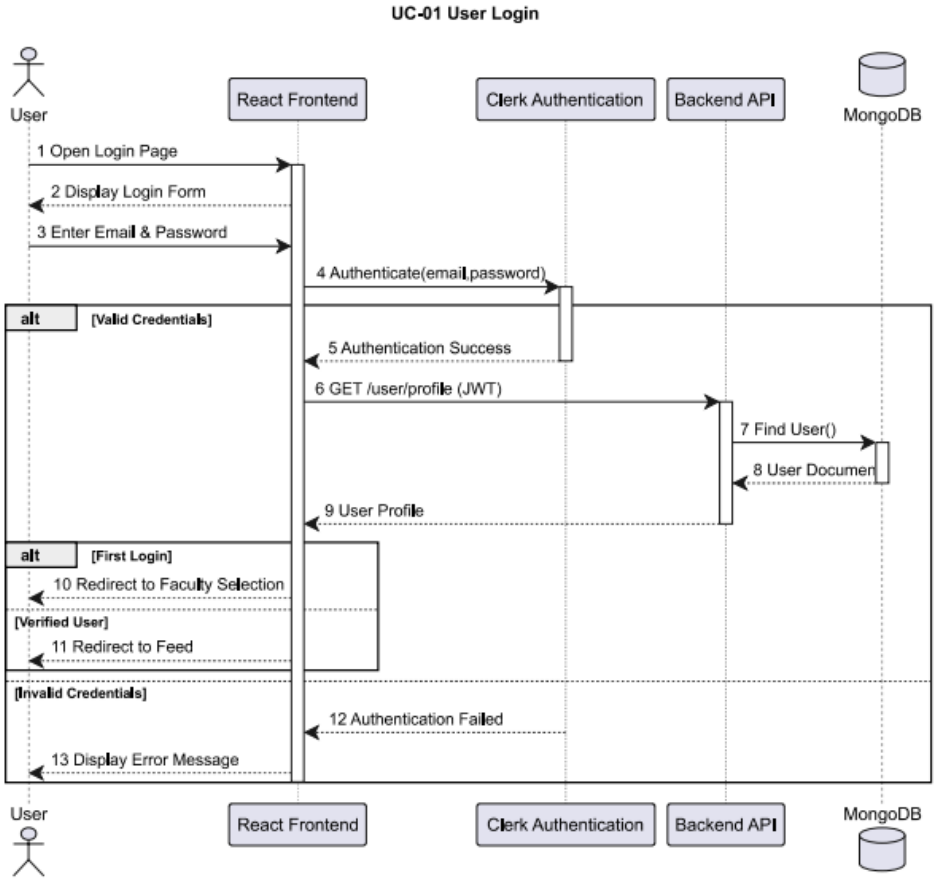

Figure 7- User Login Sequence Diagram

UC-02 — Student Verification

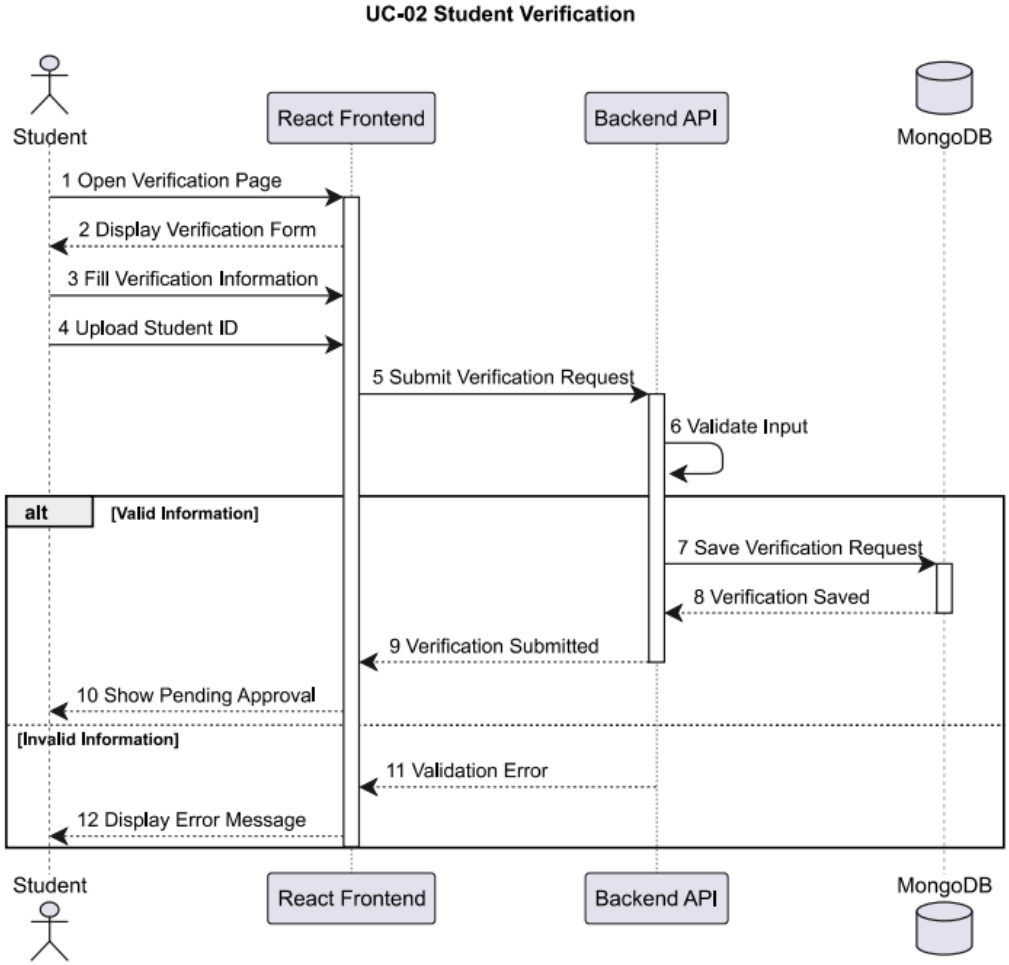

Figure 8- Seq Student Verification

UC-03 — Professor Verification

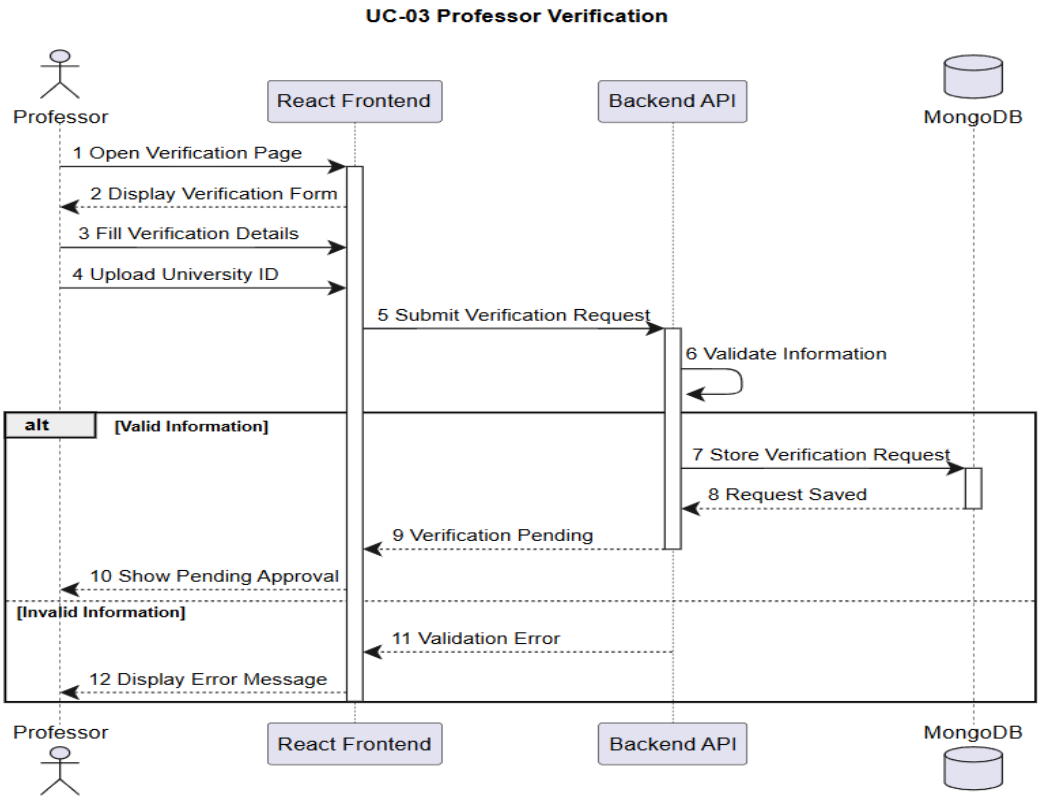

Figure 9- Seq Professor Verification

UC-04 — Faculty Selection

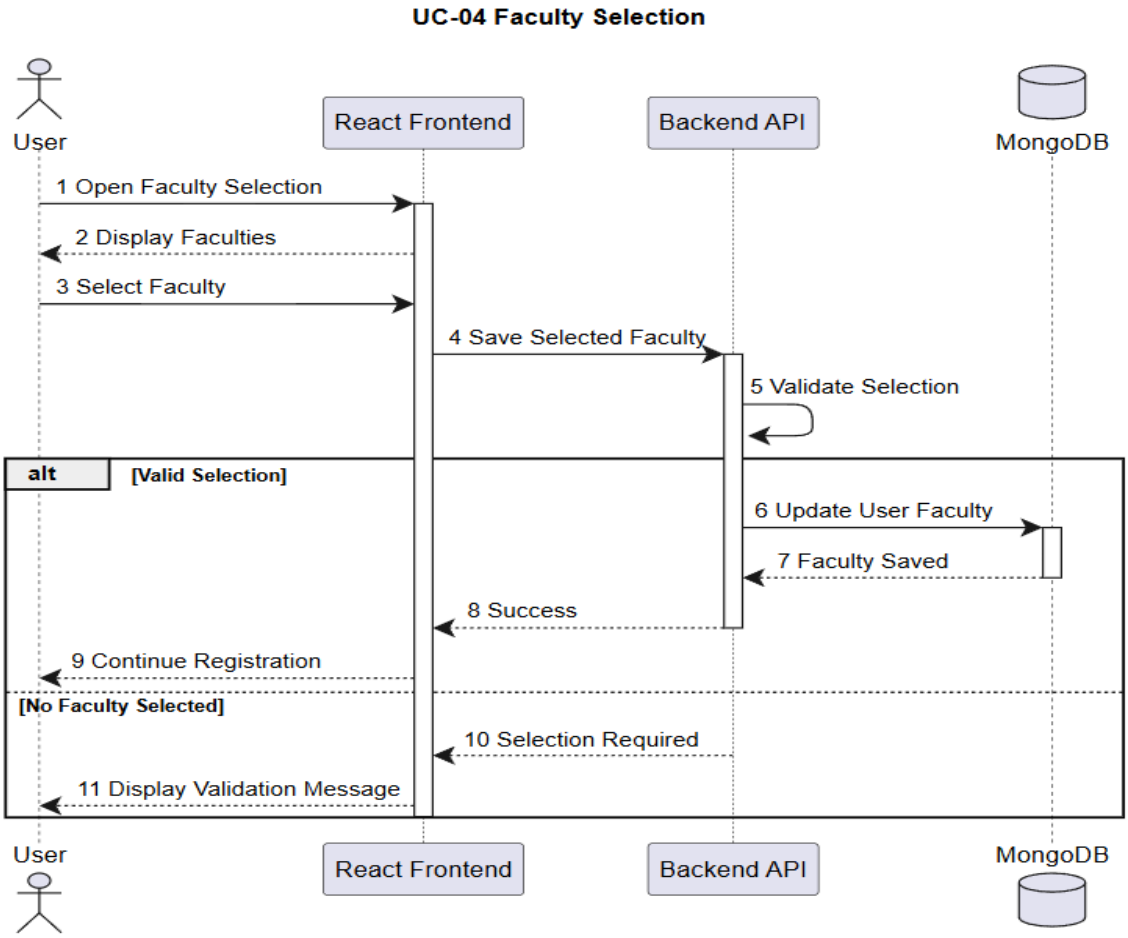

Figure 10- Seq Faculty Selection

UC-05 — Profile Management

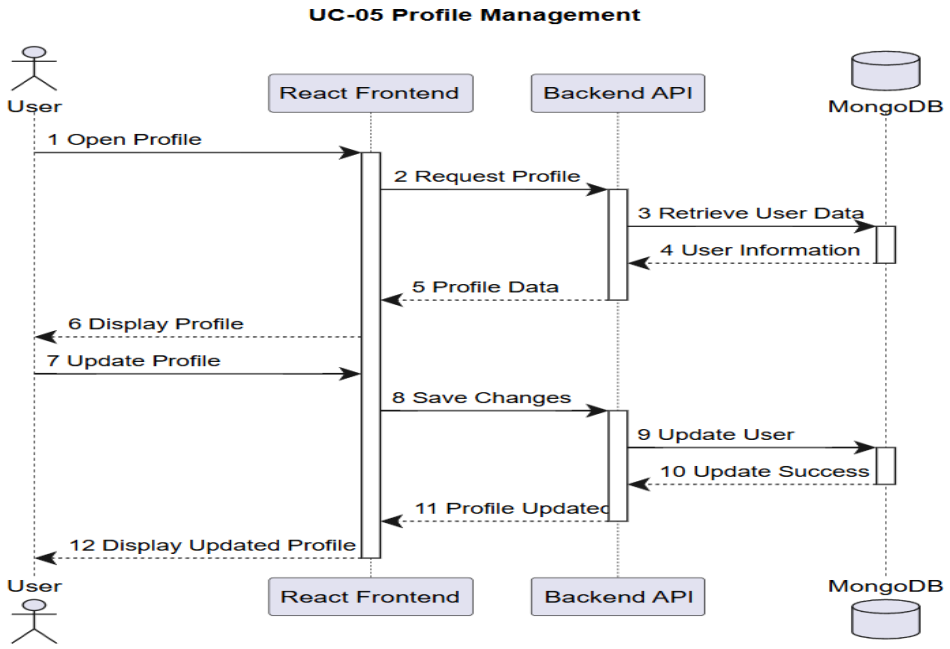

Figure 11- Seq Profile Management

UC-06 — Create Post

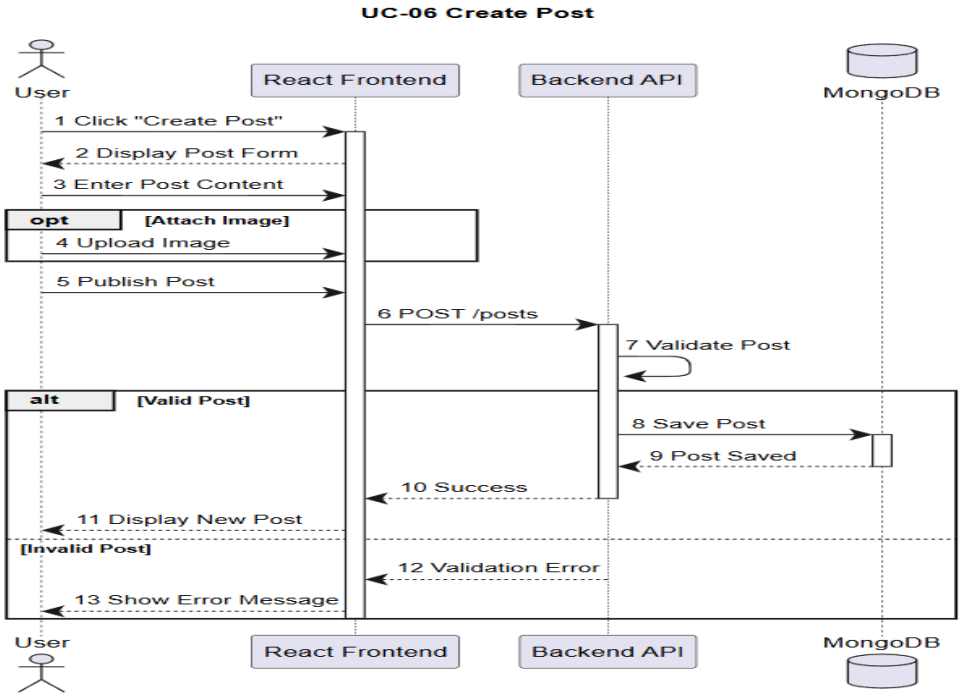

Figure 12- Seq Create Post

UC-07 — Comment Management

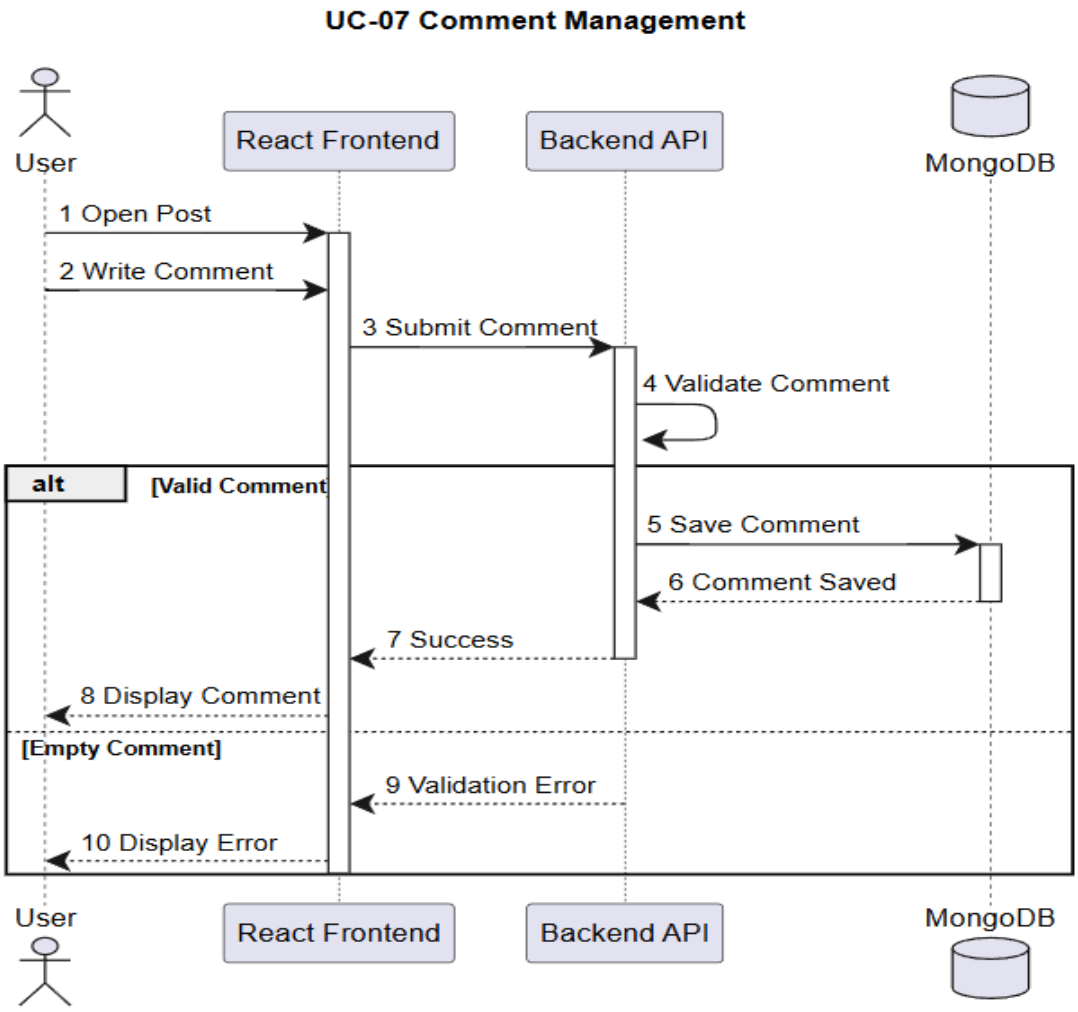

Figure 13- Seq Comment Management

UC-08 — Like Post

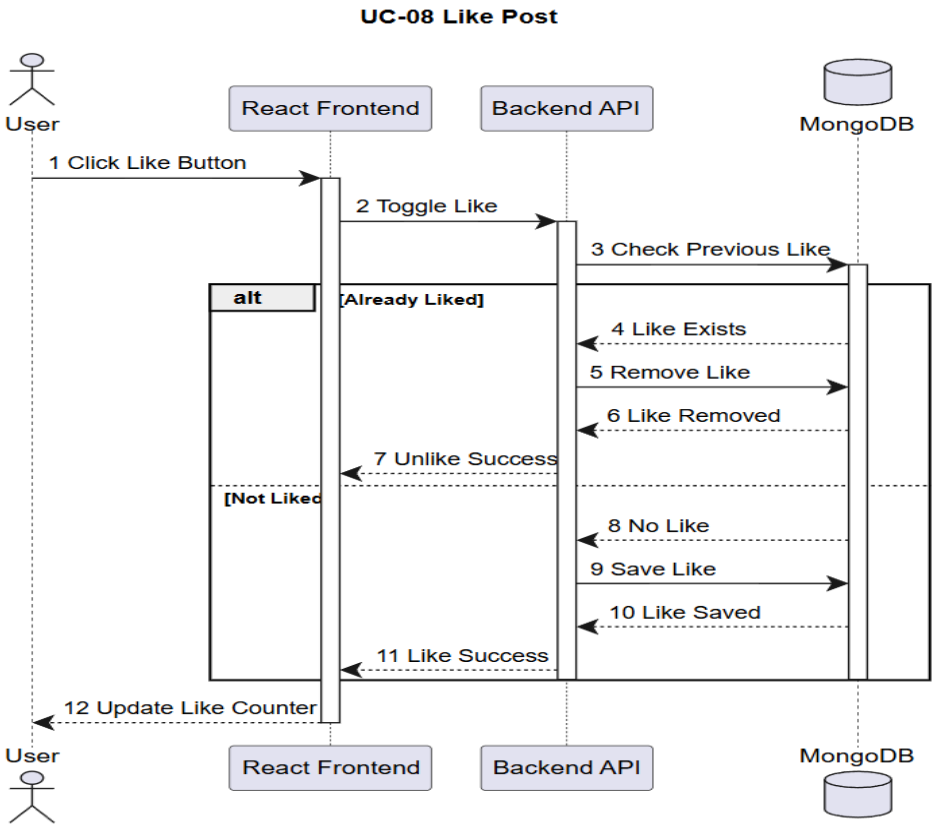

Figure 14- Seq Like Post

UC-09 — Connection Management

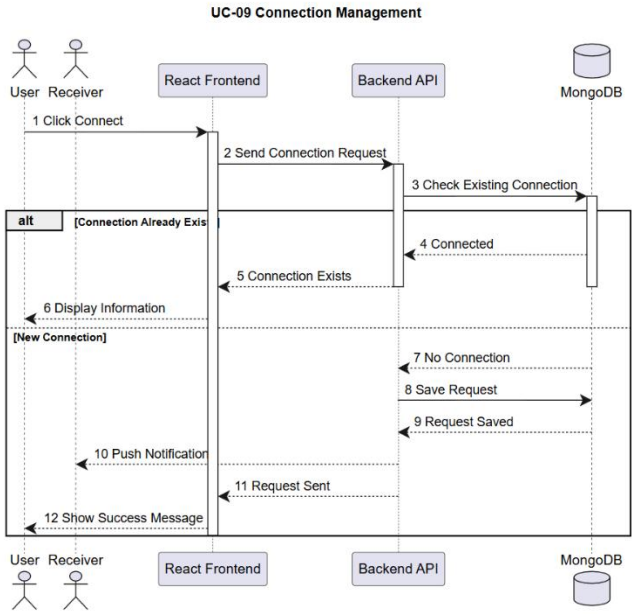

Figure 15- Seq Connection Management

UC-10 — Real-Time Messaging

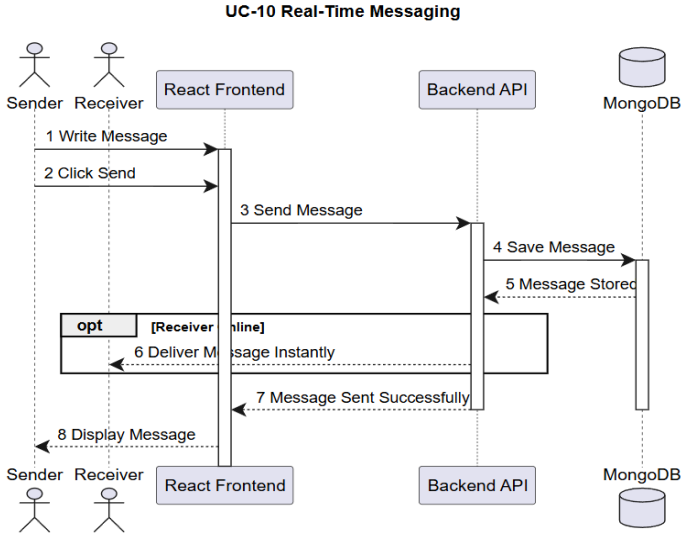

Figure 16- Seq Real-Time Messaging

UC-11 — Notification Management

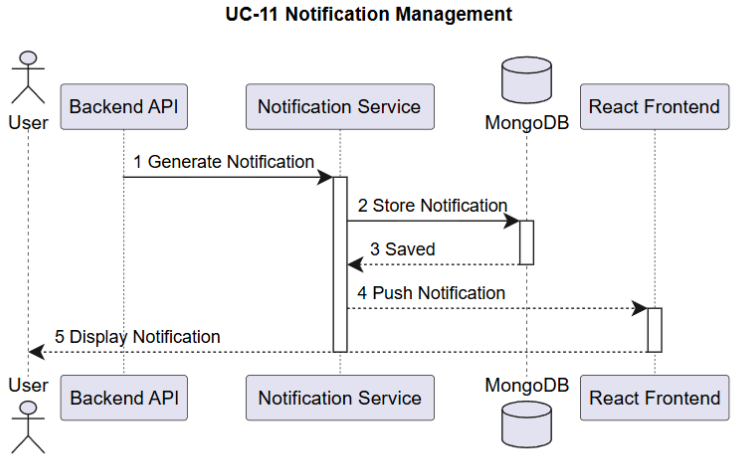

Figure 17- Seq Notification Management

UC-12 — Search Users

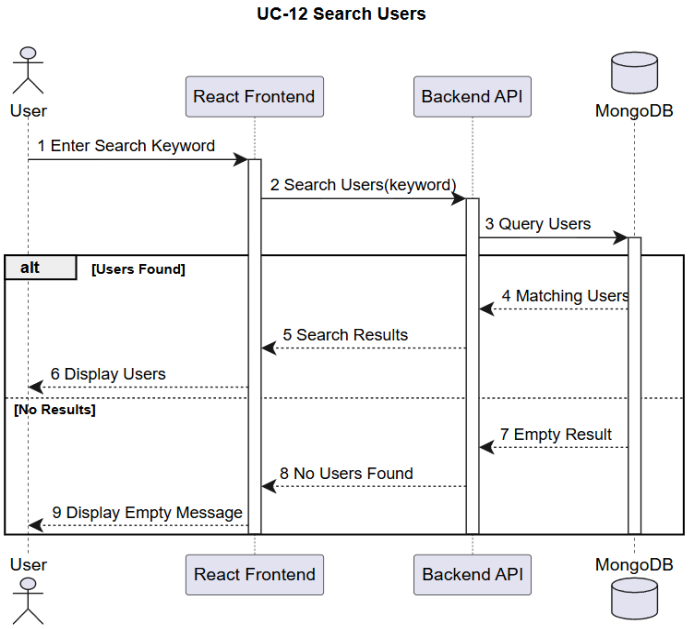

Figure 18- Seq Search Users

UC-13 — View Feed

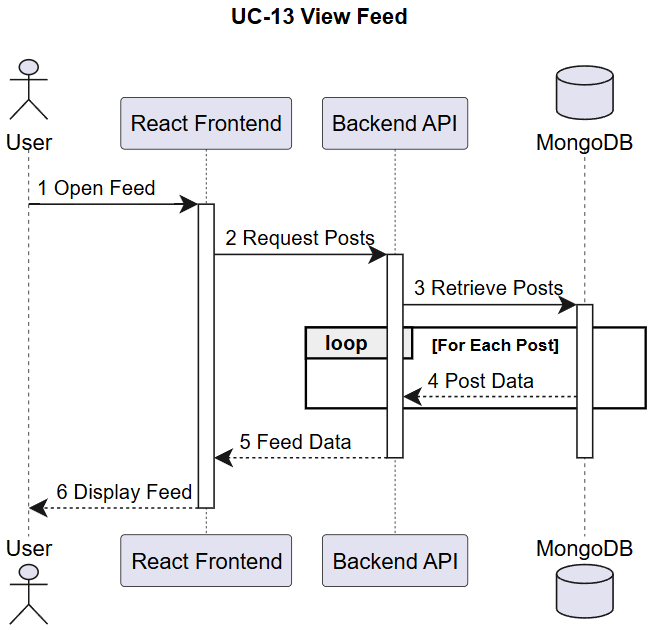

Figure 19- Seq View Feed

UC-14 — View User Profile

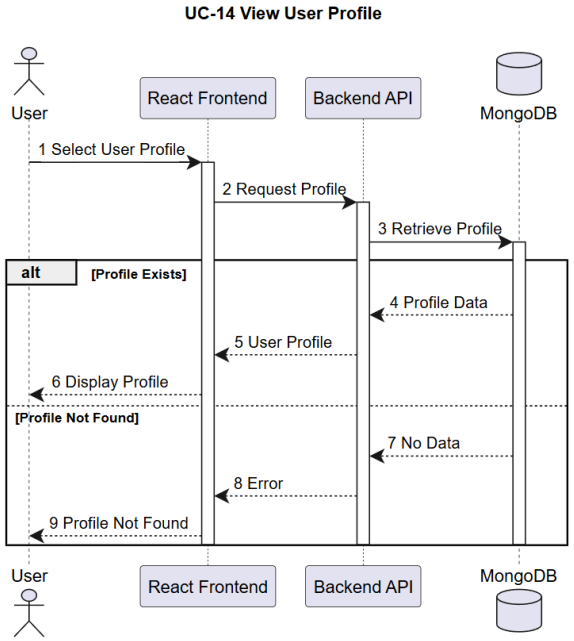

Figure 20- Seq View User Profile

UC-15 — Report Post

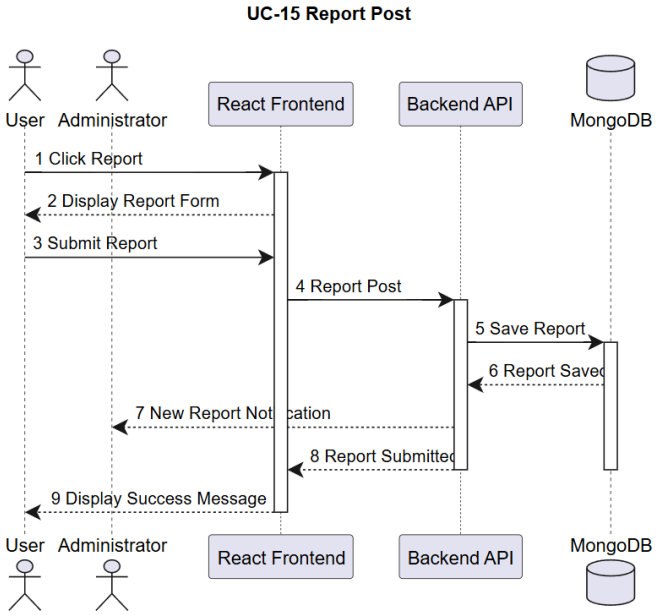

Figure 21- Seq Report Post

UC-16 — Review Reports (Administrator)

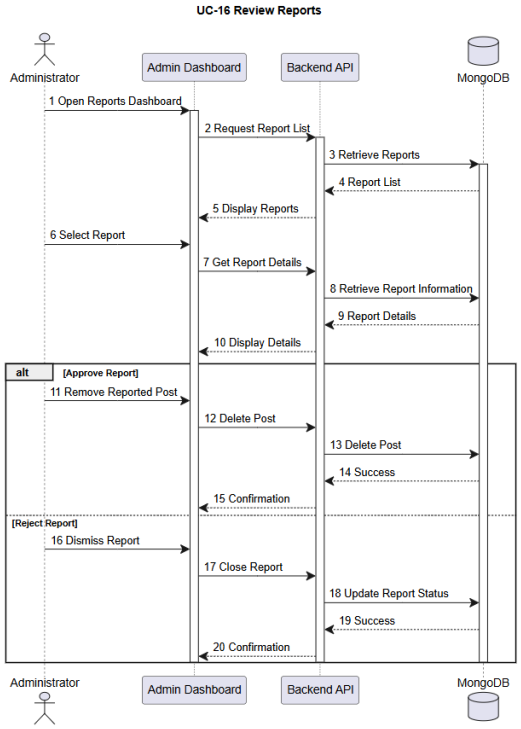

Figure 22- Seq Review Reports (Administrator)

UC-17 — Logout

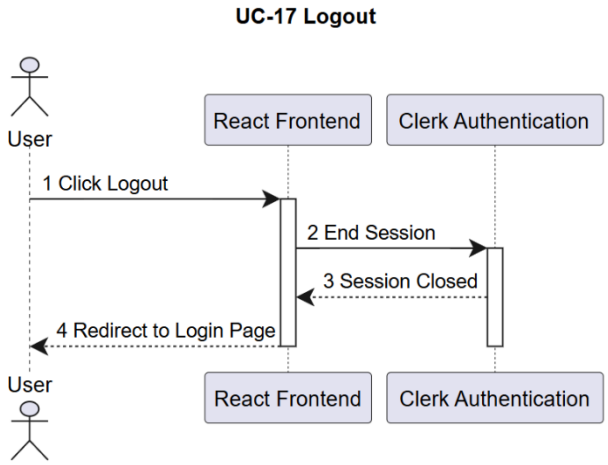

Figure 23- Seq Logout

UC-18 — View Notifications

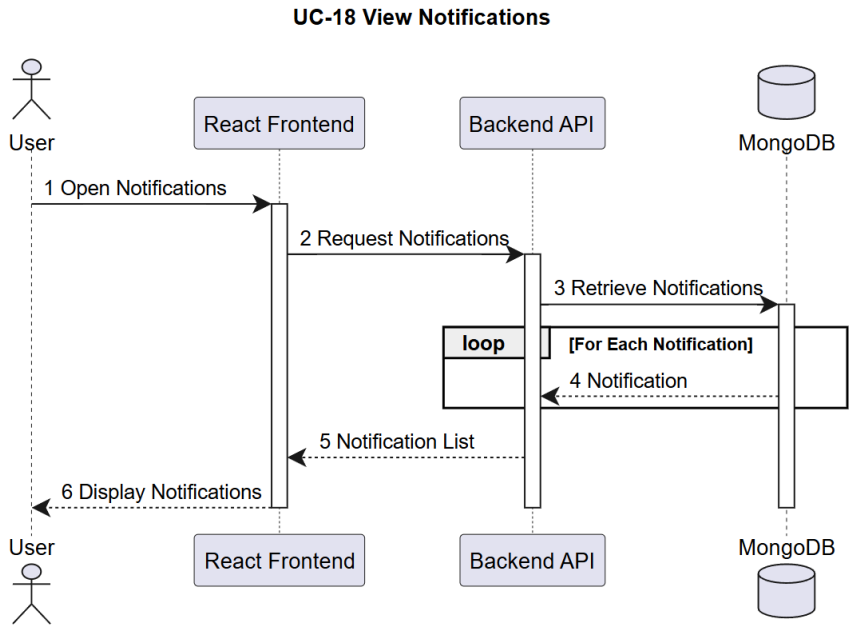

Figure 24- Seq View Notifications

UC-19 — Manage Verification Requests

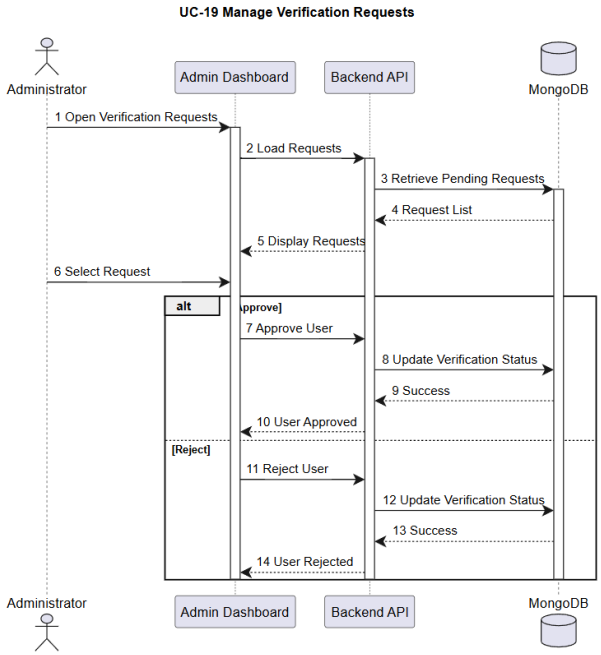

Figure 25- Seq Manage Verification Requests

UC-20 — Manage User Sessions

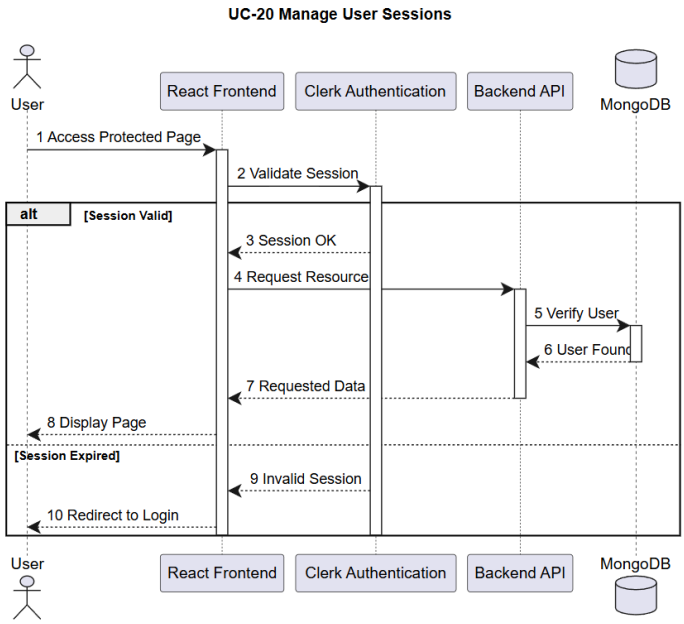

Figure 26- Seq Manage User Sessions

3.5 Activity Diagrams

3.5 Introduction

Activity diagrams are behavioral UML diagrams that illustrate the workflow of system processes from the beginning of an activity until its completion. They represent the sequence of actions, decision points, parallel activities, and alternative execution paths that occur while performing a specific system function. These diagrams provide a clear understanding of the operational logic behind each use case and simplify the analysis of complex business processes.

In the UniSphere platform, activity diagrams were developed for the major system functionalities, including authentication, profile management, content creation, messaging, notifications, connection management, reporting, and administrative operations. Each activity diagram corresponds to a previously defined use case and describes the complete workflow performed by the user and the system.

3.5.1 Activity Diagram – User Login (UC-01)

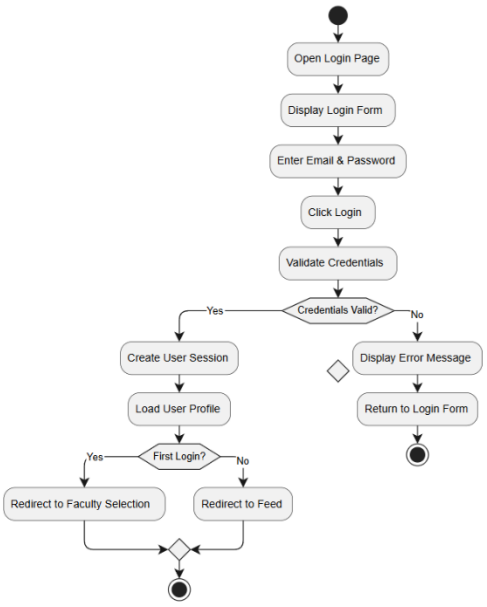

Figure 27- User Login Activity Diagram

3.5.2 Activity Diagram – Student Verification (UC-02)

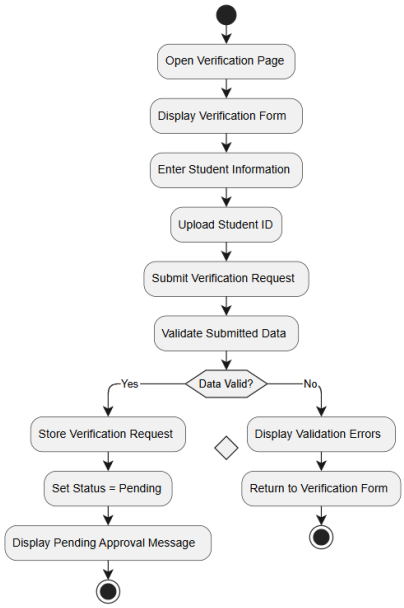

Figure 28- Activity Diagram – Student Verification

3.5.3 Activity Diagram – Professor Verification (UC-03)

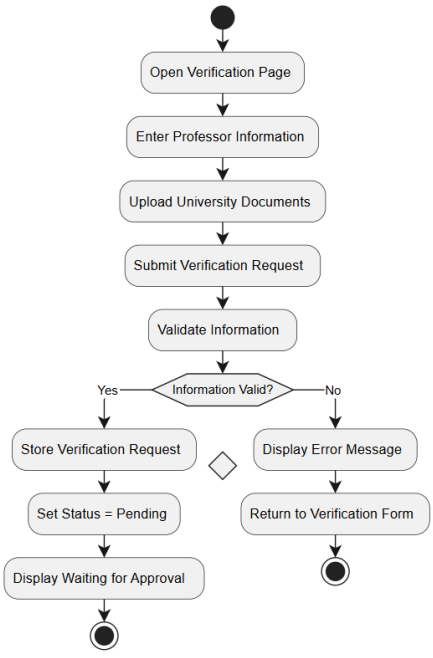

Figure 29- Activity Diagram – Professor Verification

3.5.4 Activity Diagram – Faculty Selection (UC-04)

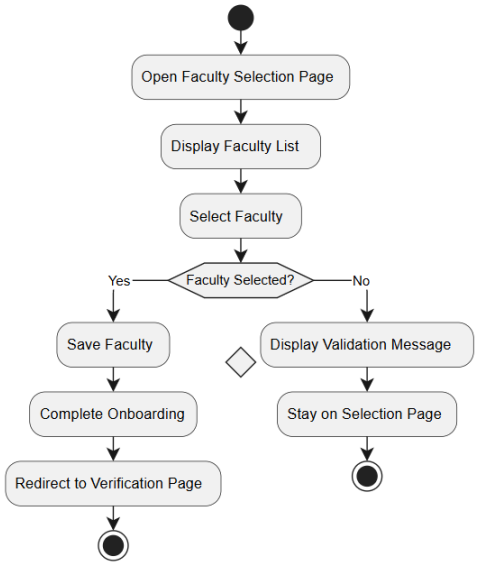

Figure 30- Activity Diagram – Faculty Selection

3.5.5 Activity Diagram – Profile Management (UC-05)

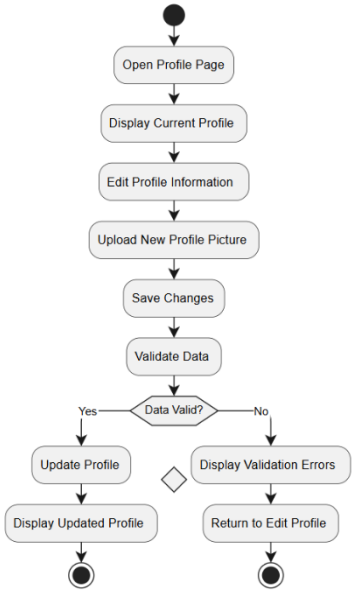

Figure 31- Activity Diagram – Profile Management

3.5.6 Activity Diagram – Create Post (UC-06)

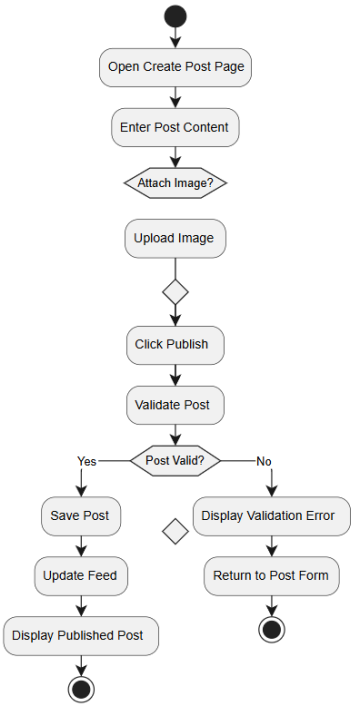

Figure 32- Activity Diagram – Create Post

3.5.7 Activity Diagram – Comment Management (UC-07)

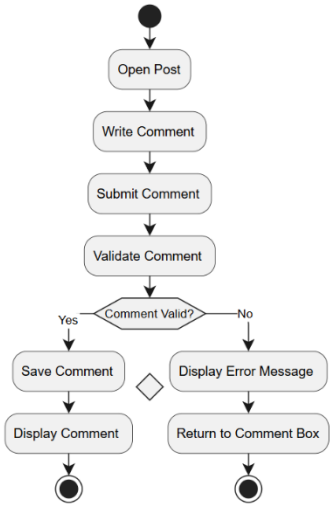

Figure 33- Activity Diagram – Comment Management

3.5.8 Activity Diagram – Like Post (UC-08)

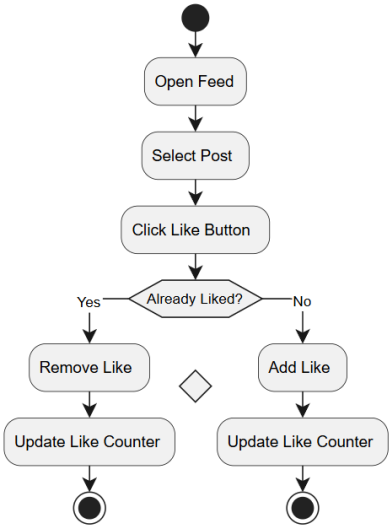

Figure 34- Activity Diagram – Like Post

3.5.9 Activity Diagram – Connection Management (UC-09)

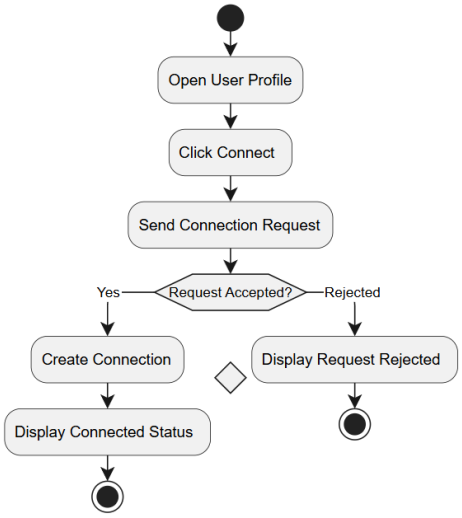

Figure 35- Activity Diagram – Connection Management

3.5.10 Activity Diagram – Real-Time Messaging (UC-10)

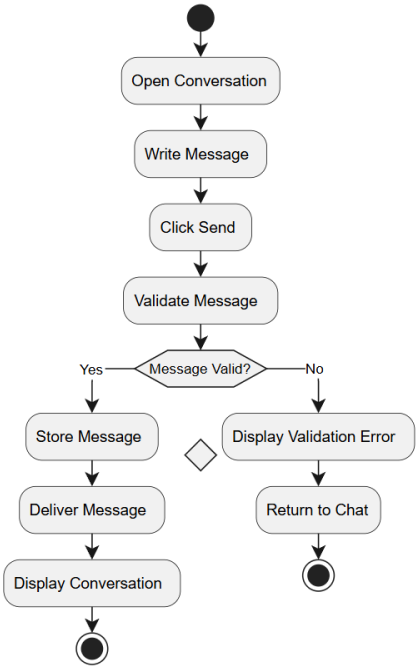

Figure 36- Activity Diagram – Real-Time Messaging

3.5.11 Activity Diagram – Notification Management (UC-11)

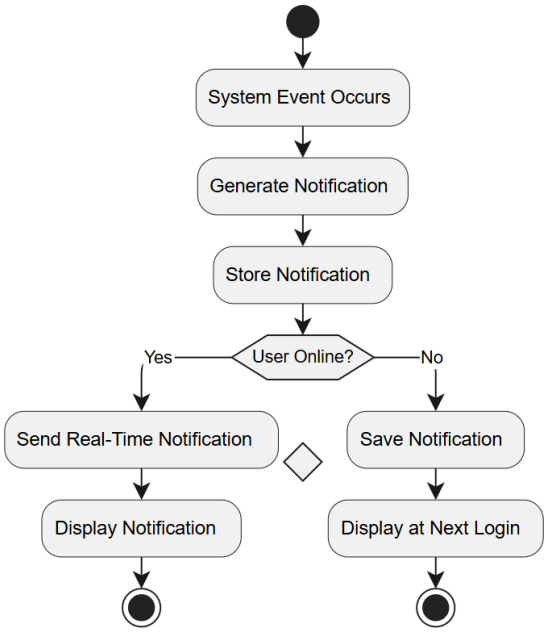

Figure 37- Activity Diagram – Notification Management

3.5.12 Activity Diagram – Search Users (UC-12)

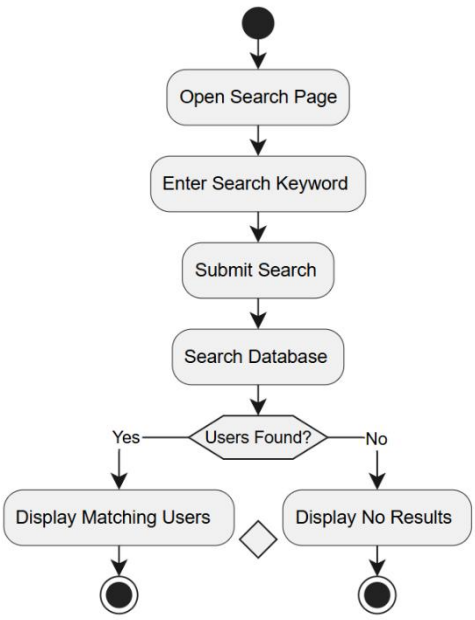

Figure 38- Activity Diagram – Search Users

3.5.13 Activity Diagram – View Feed (UC-13)

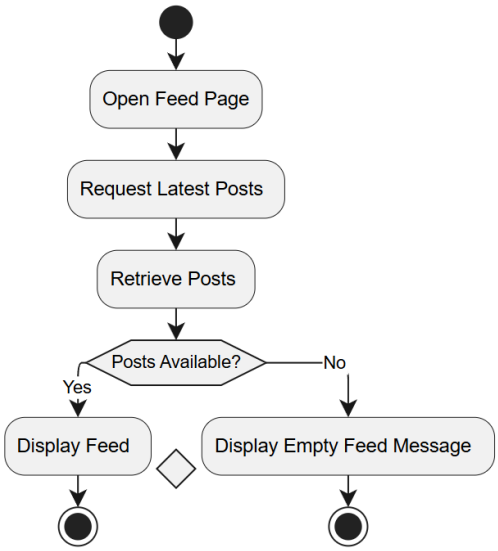

Figure 39- Activity Diagram – View Feed

3.5.14 Activity Diagram – View User Profile (UC-14)

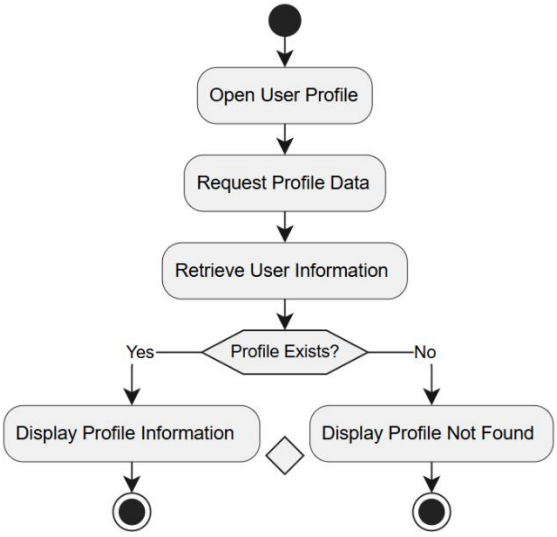

Figure 40- Activity Diagram – View User Profile

3.5.15 Activity Diagram – Report Post (UC-15)

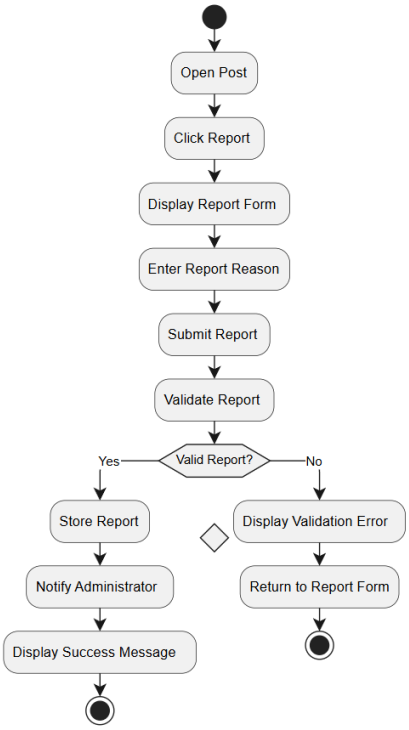

Figure 41- Activity Diagram – Report Post

3.5.16 Activity Diagram – Review Reports (UC-16)

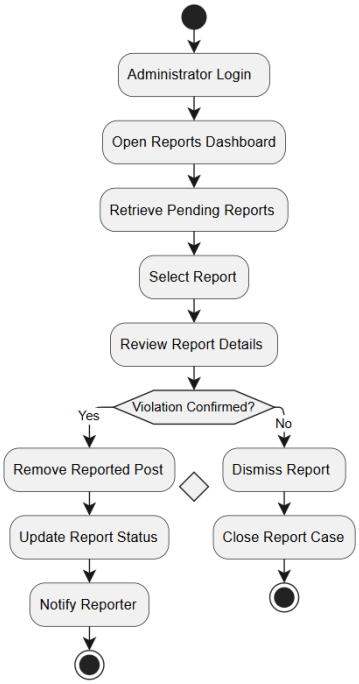

Figure 42- Activity Diagram – Review Reports

3.5.17 Activity Diagram – Logout (UC-17)

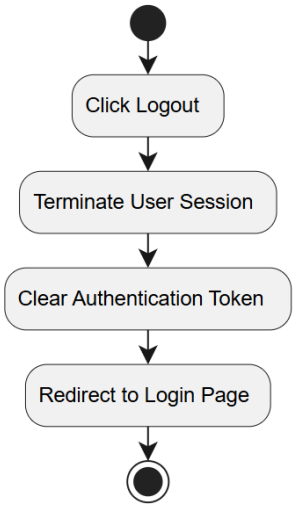

Figure 43- Activity Diagram – Logout

3.5.18 Activity Diagram – View Notifications (UC-18)

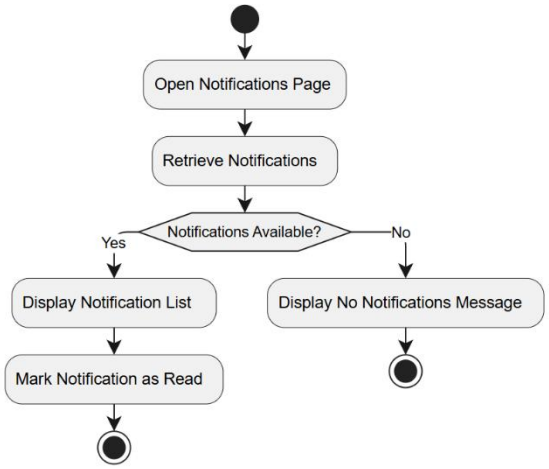

Figure 44- Activity Diagram – View Notifications

3.5.19 Activity Diagram – Manage Verification Requests (UC-19)

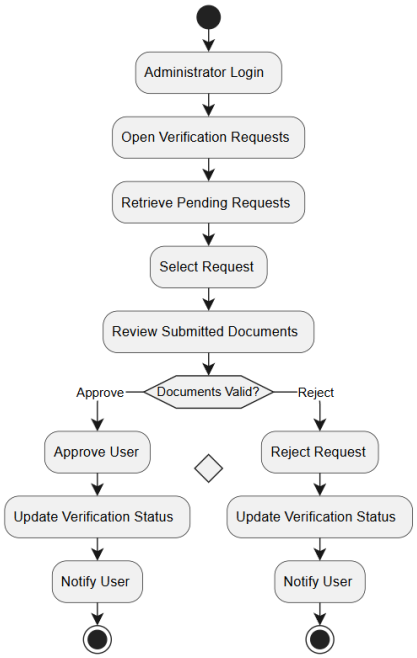

Figure 45- Activity Diagram – Manage Verification Requests

3.5.20 Activity Diagram – Manage User Sessions (UC-20)

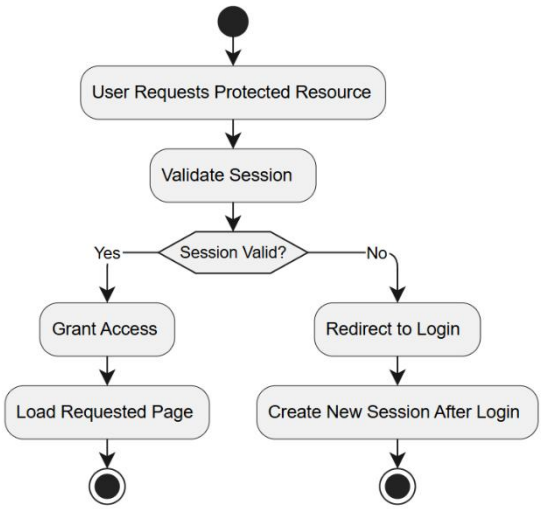

Figure 46- Activity Diagram – Manage User Sessions

3.6 Database Design

3.6 Introduction

The database is one of the core components of the UniSphere platform, as it is responsible for storing, organizing, and managing all application data. Since the system handles various types of information, including user accounts, academic posts, comments, messages, notifications, verification requests, reports, and social connections, an efficient and scalable database design is essential to ensure high performance and data consistency.

The **UniSphere** platform adopts **MongoDB**, a NoSQL document-oriented database, due to its flexibility in handling semi-structured data, its ability to scale horizontally, and its seamless integration with the MERN technology stack. Each major system entity is represented as a separate collection, while relationships between entities are maintained using document references based on ObjectId values. This design minimizes data redundancy and improves the efficiency of data retrieval and management.

3.6.1 Entity Relationship Diagram (ERD)

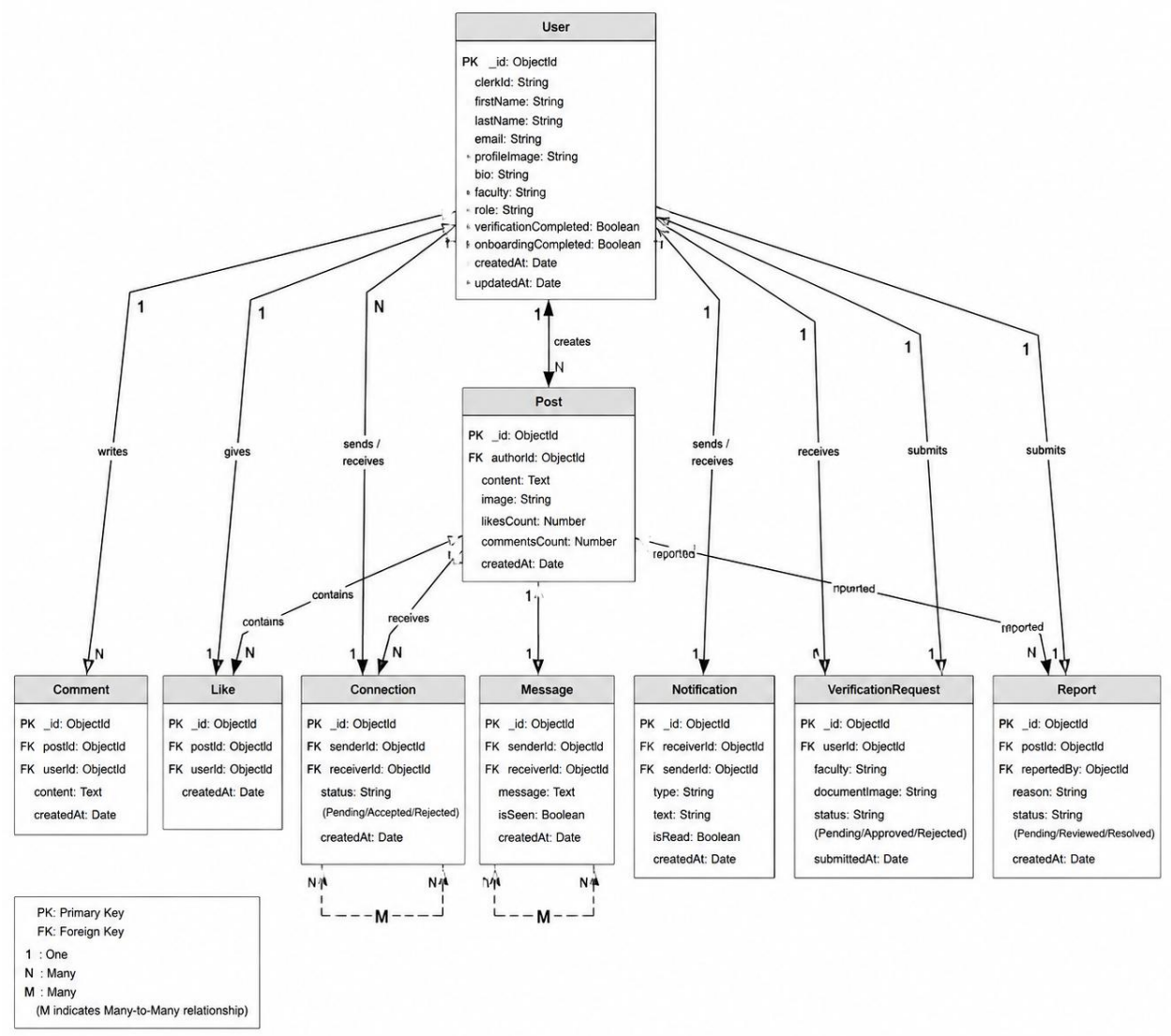

Figure 47- Entity Relationship Diagram (ERD) of the UniSphere Platform

The Entity Relationship Diagram (ERD) illustrates the logical structure of the **UniSphere** database. Although MongoDB is a NoSQL database that stores data as JSON-like documents rather than relational tables, the ERD provides a clear representation of the main collections and the relationships among them. It demonstrates how user accounts interact with posts, comments, likes, messages, notifications, verification requests, reports, and social connections, ensuring efficient organization and retrieval of system data.

3.6.2 Collections Description

Collection Name	Description
Users	Stores all user information including personal details, authentication identifiers, faculty, role, profile image, verification status, and onboarding status.
Posts	Stores all academic posts created by users, including text content, images, likes count, comments count, and creation date.
Comments	Stores comments made by users on posts, including the associated user, post, comment content, and timestamp.
Likes	Records user reactions to posts and prevents duplicate likes by maintaining user-post relationships.
Connections	Stores friendship/connection requests between users and tracks their current status (Pending, Accepted, or Rejected).
Messages	Stores private chat messages exchanged between connected users, including sender, receiver, message content, read status, and timestamp.
Notifications	Stores all system notifications such as new messages, connection requests, comments, likes, and verification updates.
VerificationRequests	Stores identity verification requests submitted by professors, including uploaded documents, faculty information, and approval status.
Reports	Stores reports submitted against inappropriate posts, including the reporting user, reported post, reason, review status, and submission date.

Figure 48- MongoDB Collections Used in the UniSphere Platform

Each collection is designed to represent a specific functional component of the **UniSphere** platform. The separation of data into independent collections improves maintainability, reduces redundancy, and allows the system to efficiently retrieve and update information. Relationships between collections are implemented using MongoDB ObjectId references, enabling flexible document-based data management while preserving data consistency across the platform.

3.6.3 Relationships Between Collections

Table 23 -Relationships Between MongoDB Collections

Source Collection	Target Collection	Relationship	Description
Users	Posts	One-to-Many (1:N)	One user can create multiple posts, while each post belongs to one user.
Users	Comments	One-to-Many (1:N)	A user can write many comments, but each comment is written by only one user.
Posts	Comments	One-to-Many (1:N)	A post can contain multiple comments, while each comment belongs to one post.
Users	Likes	One-to-Many (1:N)	A user can like multiple posts.
Posts	Likes	One-to-Many (1:N)	A post can receive likes from multiple users.
Users	Connections	One-to-Many (1:N)	A user can send and receive multiple connection requests.
Users	Messages	One-to-Many (1:N)	A user can send and receive many private messages.
Users	Notifications	One-to-Many (1:N)	A user can receive multiple system notifications.

Source Collection	Target Collection	Relationship	Description
Users	VerificationRequests	One-to-Many (1:N)	A user can submit verification requests during the verification process.
Posts	Reports	One-to-Many (1:N)	A post may receive multiple reports from different users.
Users	Reports	One-to-Many (1:N)	A user can submit multiple reports against inappropriate posts.

Relationship Notes

- **Users ↔ Posts:** One user may create multiple posts.
- **Users ↔ Comments:** One user may publish multiple comments.
- **Posts ↔ Comments:** One post may contain many comments.
- **Users ↔ Likes ↔ Posts:** Likes act as a bridge collection between users and posts, representing a **many-to-many** interaction conceptually while being implemented through references.
- **Users ↔ Connections:** Each connection references two users (sender and receiver), allowing users to establish multiple social relationships.
- **Users ↔ Messages:** Messages reference both sender and receiver, enabling real-time communication.
- **Users ↔ Notifications:** Notifications are associated with the receiving user and optionally reference the sender.
- **Users ↔ VerificationRequests:** Each verification request belongs to one user and stores the submitted verification information.
- **Posts ↔ Reports:** Multiple users can report the same post, and each report references both the reported post and the reporting user.

The use of ObjectId references instead of embedded documents provides better scalability and minimizes data duplication. This approach enables efficient querying, easier maintenance, and supports future expansion of the **UniSphere** platform without significant modifications to the database structure.

3.6.4 Justification for Using MongoDB

The UniSphere platform uses **MongoDB** as its primary database management system due to its flexibility, scalability, and compatibility with modern web application development. Since UniSphere is developed using the **MERN Stack (MongoDB, Express.js, React.js, and Node.js)**, MongoDB provides seamless integration with the backend and allows efficient storage and retrieval of JSON-like documents.

Unlike traditional relational databases that require predefined schemas and complex table relationships, MongoDB adopts a document-oriented model that simplifies the management of dynamic and semi-structured data. This feature is particularly suitable for UniSphere, where user profiles, academic posts, comments, messages, notifications, verification requests, and reports may evolve over time with additional attributes.

MongoDB also offers excellent performance when handling large volumes of concurrent requests, making it appropriate for a social networking platform where many users may simultaneously create posts, exchange messages, submit comments, and receive notifications in real time. Furthermore, its native support for horizontal scaling ensures that the platform can accommodate future growth without requiring significant changes to the database architecture.

For these reasons, MongoDB provides an efficient, scalable, and maintainable solution that satisfies both the functional and non-functional requirements of the UniSphere platform while supporting future enhancements and increased user activity.

Table 24- Advantages of MongoDB in UniSphere

Advantage	Description
Flexible Schema	Allows adding new fields without modifying existing database structures.
High Performance	Provides fast read and write operations for real-time interactions.
Horizontal Scalability	Supports future system expansion by distributing data across multiple servers.
MERN Stack Compatibility	Integrates naturally with Node.js and JavaScript objects.
JSON Document Storage	Stores data in BSON documents that closely match JavaScript objects.
Easy Maintenance	Simplifies database updates and application development.
Reduced Data Redundancy	Uses ObjectId references to efficiently relate collections.
Suitable for Social Platforms	Efficiently handles posts, comments, chats, notifications, and user-generated content.

3.7 Class Diagram

3.7 Introduction

The Class Diagram represents the static structure of the UniSphere platform by illustrating the main software classes, their attributes, methods, and relationships. Unlike the Entity Relationship Diagram, which focuses on database collections, the Class Diagram describes the object-oriented architecture implemented in the

application and demonstrates how different software components interact to accomplish the system's functionality.

The UniSphere platform follows the **Model–View–Controller (MVC)** architectural pattern on the backend. Therefore, the Class Diagram primarily represents the **Model layer**, which defines the application's business entities, while also illustrating their associations with other system components. This design improves modularity, maintainability, and scalability by separating responsibilities among different classes.

3.7.1 Class Diagram

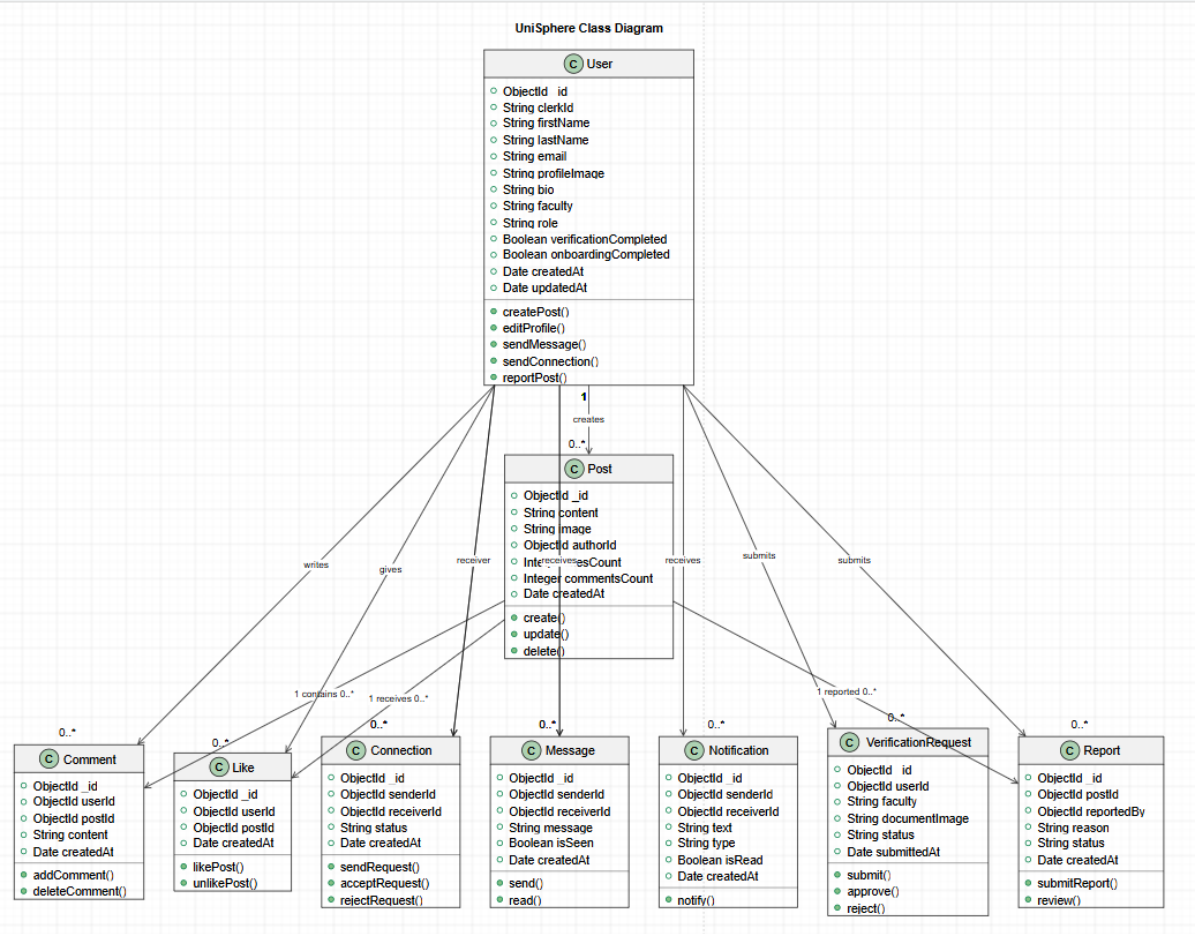

Figure 49 - Class Diagram of the UniSphere Platform

The Class Diagram illustrates the principal classes implemented within the UniSphere platform. Each class encapsulates a specific responsibility and contains a set of attributes and operations that support the system's business logic. Relationships such as associations and dependencies demonstrate how users interact with posts, comments, messages, notifications, verification requests, reports, and social connections throughout the platform.

3.7.2 Description of Main Classes

Table 25- Main Classes of the UniSphere Platform

Class Name	Responsibility
User	Represents the registered user and stores personal information, authentication details, profile information, faculty, and verification status.
Post	Represents academic posts published by users, including content, media, reactions, and timestamps.
Comment	Represents comments associated with posts and their corresponding authors.
Like	Represents user reactions to posts and maintains the relationship between users and liked posts.
Connection	Manages social relationships and connection requests between users.
Message	Represents private conversations exchanged between connected users.
Notification	Represents system-generated notifications related to user activities.
VerificationRequest	Stores professor verification requests and uploaded supporting documents.

Class Name	Responsibility
Report	Represents reports submitted against inappropriate posts for administrative review.

3.7.3 Relationships Between Classes

Table 26- Relationships Between Main Classes

Source Class	Target Class	Relationship	Description
User	Post	Association (1:N)	A user can create multiple posts.
User	Comment	Association (1:N)	A user can write multiple comments.
Post	Comment	Aggregation (1:N)	A post contains multiple comments.
User	Like	Association (1:N)	A user can like multiple posts.
Post	Like	Aggregation (1:N)	A post may receive multiple likes.
User	Connection	Association (1:N)	Users can establish multiple social connections.
User	Message	Association (1:N)	Users can send and receive multiple private messages.
User	Notification	Association (1:N)	A user can receive multiple notifications.

Source Class	Target Class	Relationship	Description
User	VerificationRequest	Association (1:N)	Users can submit verification requests.
User	Report	Association (1:N)	Users can report multiple posts.
Post	Report	Association (1:N)	A post may receive multiple reports.

3.7.4 Design Justification

The object-oriented design adopted in UniSphere promotes modularity by encapsulating related data and behaviors within dedicated classes. Each class has a single responsibility, reducing coupling while increasing cohesion across the system. The relationships between classes reflect the real-world interactions of the academic social platform and facilitate future maintenance, testing, and feature expansion.

Furthermore, the Class Diagram closely aligns with the MVC architecture implemented in the backend. The Model classes define the application's core business entities, while controllers manage user requests and coordinate interactions between the models and the user interface. This separation of concerns simplifies code maintenance and improves the overall scalability and reliability of the platform.

3.8 Test Cases

3.8 Introduction

Test cases are developed to verify that each functional requirement of the **UniSphere** platform operates as expected. Each test case specifies the testing objective, required preconditions, testing steps, expected results, and execution status. The test cases were designed according to the functional requirements

identified during the analysis phase and the use case specifications presented earlier in this chapter.

The primary objective of these test cases is to ensure that every major function of the system behaves correctly under normal operating conditions while validating user interactions, system responses, and data consistency throughout the platform.

Table 27 - Test Cases Table

TC ID	Requirement ID	Test Case Name	Preconditions	Test Steps	Expected Result	Status
TC-01	FR-01	User Login	Registered user exists	Enter valid email and password then click Login	User is authenticated and redirected successfully	Passed
TC-02	FR-02	Student Verification	Student account exists	Submit verification information	Verification request is stored successfully	Passed
TC-03	FR-03	Professor Verification	Professor account exists	Submit professor verification request	Request status becomes Pending	Passed
TC-04	FR-04	Faculty Selection	User logged in لأول مرة	Select a faculty and continue	Selected faculty is saved successfully	Passed
TC-05	FR-05	Profile Management	User authenticated	Update profile information	Profile information is updated successfully	Passed

TC ID	Requirement ID	Test Case Name	Preconditions	Test Steps	Expected Result	Status
TC-06	FR-06	Create Post	User authenticated	Create and publish a post	Post appears in the news feed	Passed
TC-07	FR-07	Comment Management	Existing post available	Add a comment	Comment appears under the selected post	Passed
TC-08	FR-08	Like Post	Existing post available	Click Like button	Like counter is updated	Passed
TC-09	FR-09	Connection Management	Two registered users	Send connection request	Request is delivered successfully	Passed
TC-10	FR-10	Real-Time Messaging	Users connected	Send message	Receiver gets message instantly	Passed
TC-11	FR-11	Notification Management	User authenticated	Trigger system notification	Notification appears immediately	Passed
TC-12	FR-12	Search Users	User authenticated	Search by user name	Matching users are displayed	Passed
TC-13	FR-13	View Feed	User authenticated	Open Feed page	Latest posts are displayed	Passed
TC-14	FR-14	View User Profile	User authenticated	Open another user's profile	Profile information is displayed	Passed

TC ID	Requirement ID	Test Case Name	Preconditions	Test Steps	Expected Result	Status
TC-15	FR-15	Report Post	Existing post available	Report inappropriate post	Report is stored successfully	Passed
TC-16	FR-16	Review Reports	Administrator authenticated	Review reported posts	Administrator processes reports	Passed
TC-17	FR-17	Logout	User authenticated	Click Logout	User session ends successfully	Passed
TC-18	FR-18	View Notifications	User authenticated	Open Notifications page	Notification history is displayed	Passed
TC-19	FR-19	Manage Verification Requests	Administrator authenticated	Approve or reject verification	User status is updated	Passed
TC-20	FR-20	Manage User Sessions	Active session exists	Access protected page	Session validated successfully	Passed

3.9 Requirements Traceability Matrix (RTM)

The **Requirements Traceability Matrix (RTM)** is used to establish and maintain traceability between the functional requirements of the UniSphere platform and their corresponding design and testing artifacts. It ensures that each identified requirement has been represented as a use case, modeled through Sequence and Activity Diagrams, and validated through an associated test case.

The RTM also provides a systematic mechanism for verifying requirement coverage and identifying potential gaps between the analysis, design,

implementation, and testing phases. By maintaining traceability throughout the development lifecycle, the matrix demonstrates that the major functional requirements defined for the UniSphere platform have been addressed and tested.

Table 28 - Requirements Traceability Matrix (RTM)

Requirement ID	Requirement Name	Use Case ID	Sequence Diagram	Activity Diagram	Test Case ID	Status
FR-01	User Login	UC-01	SD-01	AD-01	TC-01	Verified
FR-02	Student Verification	UC-02	SD-02	AD-02	TC-02	Verified
FR-03	Professor Verification	UC-03	SD-03	AD-03	TC-03	Verified
FR-04	Faculty Selection	UC-04	SD-04	AD-04	TC-04	Verified
FR-05	Profile Management	UC-05	SD-05	AD-05	TC-05	Verified
FR-06	Create Post	UC-06	SD-06	AD-06	TC-06	Verified
FR-07	Comment Management	UC-07	SD-07	AD-07	TC-07	Verified
FR-08	Like Post	UC-08	SD-08	AD-08	TC-08	Verified
FR-09	Connection Management	UC-09	SD-09	AD-09	TC-09	Verified
FR-10	Real-Time Messaging	UC-10	SD-10	AD-10	TC-10	Verified
FR-11	Notification Management	UC-11	SD-11	AD-11	TC-11	Verified
FR-12	Search Users	UC-12	SD-12	AD-12	TC-12	Verified
FR-13	View Feed	UC-13	SD-13	AD-13	TC-13	Verified
FR-14	View User Profile	UC-14	SD-14	AD-14	TC-14	Verified

Requirement ID	Requirement Name	Use Case ID	Sequence Diagram	Activity Diagram	Test Case ID	Status
FR-15	Report Post	UC-15	SD-15	AD-15	TC-15	Verified
FR-16	Review Reports	UC-16	SD-16	AD-16	TC-16	Verified
FR-17	Logout	UC-17	SD-17	AD-17	TC-17	Verified
FR-18	View Notifications	UC-18	SD-18	AD-18	TC-18	Verified
FR-19	Manage Verification Requests	UC-19	SD-19	AD-19	TC-19	Verified
FR-20	Manage User Sessions	UC-20	SD-20	AD-20	TC-20	Verified

RTM Analysis

As illustrated in the Requirements Traceability Matrix, each of the twenty functional requirements is mapped to a corresponding Use Case, Sequence Diagram, Activity Diagram, and Test Case. This mapping provides complete traceability across the major analysis, design, and testing artifacts used in the **UniSphere** project.

The matrix also facilitates requirement verification by allowing each system function to be traced from its original definition through its behavioral design and corresponding validation test. Consequently, changes to a requirement can be systematically reflected in its associated design models and test cases, improving maintainability and reducing the possibility of inconsistencies during future system development.

3.10 Site Map

3.10 Introduction

The Site Map provides a hierarchical representation of the navigation structure of the UniSphere platform. It illustrates the organization of the application's pages

and demonstrates how users navigate between different modules after authentication. The Site Map improves the understanding of the application's information architecture and ensures that all major system functionalities are logically connected and easily accessible.

The navigation structure was designed to provide an intuitive user experience while minimizing the number of steps required to access the platform's primary services. Depending on the user's authentication and verification status, different navigation paths are provided to ensure secure access to system resources.

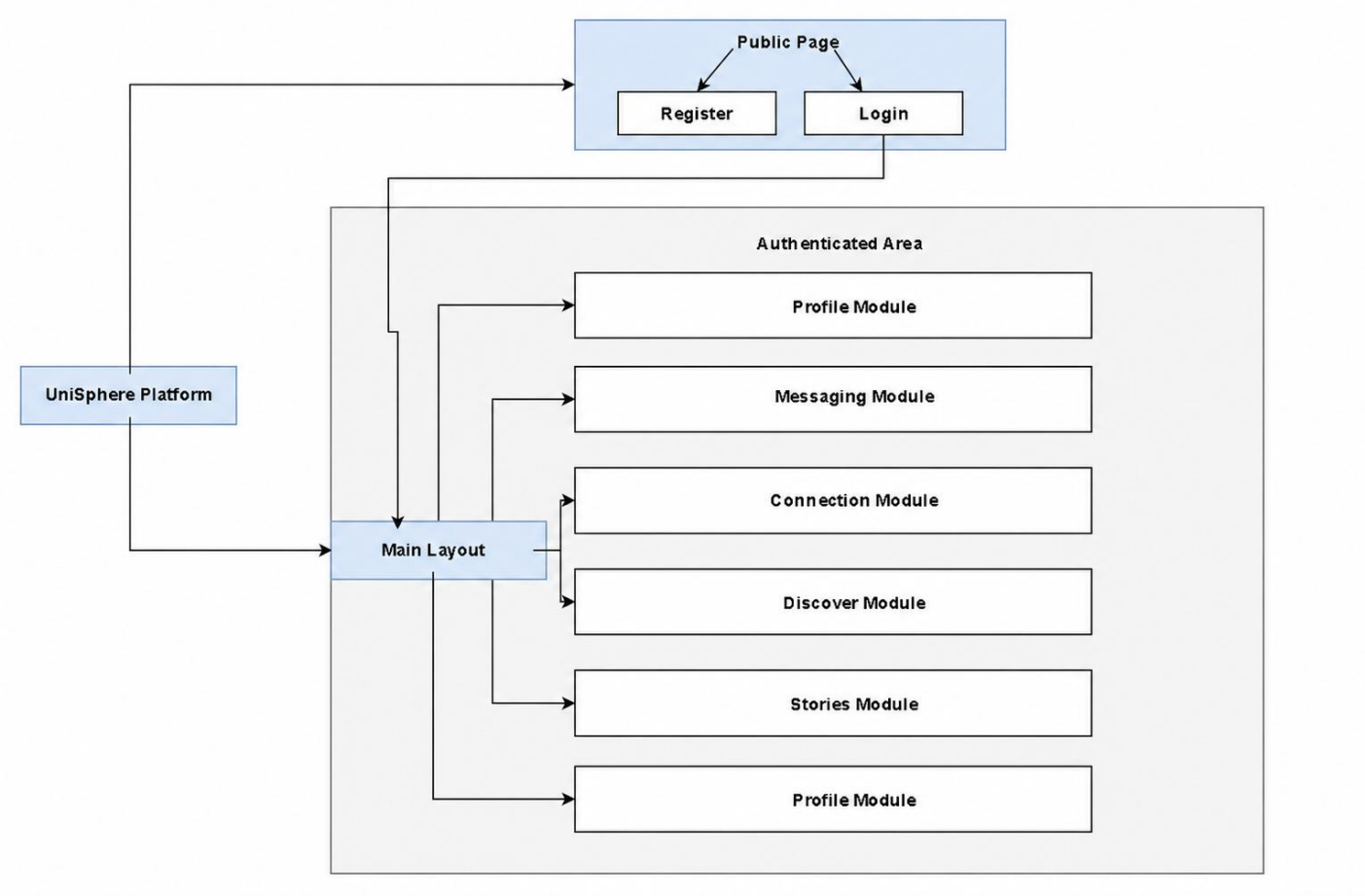

Figure 50 - Site Map of the UniSphere Platform

The Site Map illustrates the complete navigation hierarchy of the UniSphere platform. It begins with the authentication process and guides users through the onboarding and verification stages before granting access to the main system modules. Once authenticated and verified, users can navigate to academic posts,

messaging, user profiles, notifications, social connections, content discovery, and administrative functionalities according to their permissions.

3.10.1 Navigation Structure

Table 29 - Main Pages of the UniSphere Platform

Page	Description
Login	Authenticates users using Email/Password or Google SSO through Clerk Authentication.
Faculty Selection	Allows first-time users to select their faculty before accessing the platform.
Verification	Enables professors to submit identity verification requests.
Pending Approval	Displays the verification status while waiting for administrator approval.
Feed	Displays academic posts shared by students and professors.
Create Post	Allows users to create and publish academic posts with optional images.
Profile	Displays and manages the user's personal profile and academic information.
Messages	Lists all conversations with connected users.
Chat Box	Enables real-time private messaging between users.
Connections	Manages connection requests and displays existing connections.
Discover	Allows users to search and discover other students and professors.
Notifications	Displays system notifications related to messages, likes, comments, reports, and connection requests.

Page	Description
Admin Reports	Allows administrators to review reported posts and take moderation actions.

3.10.2 Site Map Design Justification

The Site Map was designed to provide a clear and organized navigation flow that aligns with the functional requirements of the UniSphere platform. The hierarchical structure separates authentication, onboarding, academic interaction, communication, and administrative management into independent modules. This organization improves maintainability, simplifies future feature expansion, and supports the overall scalability of the application while ensuring an intuitive experience for end users.

3.11 Chapter Summary

This chapter presented the complete design of the **UniSphere** platform. It introduced the system architecture through UML diagrams, including the Use Case Diagram, Sequence Diagrams, Activity Diagrams, Entity Relationship Diagram, and Class Diagram. Furthermore, the chapter described the database structure, navigation hierarchy, software test cases, and the Requirements Traceability Matrix. Together, these design artifacts provide a comprehensive blueprint for implementing the platform and ensure that all functional requirements are accurately represented before the implementation phase begins.

Chapter Four System Implementation

4.1 Introduction

This chapter presents the implementation phase of the UniSphere platform, where the system design presented in the previous chapter is transformed into a fully functional web application. The implementation process involved integrating the frontend, backend, database, authentication services, and real-time communication components to develop a complete academic social networking platform.

The UniSphere platform was implemented using the MERN Stack, consisting of **MongoDB**, **Express.js**, **React.js**, and **Node.js**, together with several supporting technologies such as **Clerk Authentication**, **Redux Toolkit**, **Tailwind CSS**, **Socket-like Server-Sent Events (SSE)**, and **ImageKit**. These technologies were selected to ensure scalability, maintainability, security, and high system performance while providing an intuitive and responsive user experience.

This chapter describes the development tools, software frameworks, programming languages, and technologies adopted during the implementation process. It also presents the graphical user interfaces of the system and explains the functionality of each major screen.

4.2 Development Tools

The implementation of the UniSphere platform required a modern development environment consisting of frontend technologies, backend frameworks, database systems, authentication services, and supporting development tools. These technologies were selected according to the functional and non-functional requirements identified during the analysis phase. Together, they provide a scalable, secure, and maintainable architecture that supports both current system requirements and future expansion.

4.2.1 Frontend Development Tools

The frontend of the **UniSphere** platform was developed using modern web technologies to provide an interactive, responsive, and user-friendly interface. React.js was selected as the primary frontend framework due to its component-based architecture and efficient rendering mechanism. Tailwind CSS was used to

design responsive interfaces with a consistent visual identity, while Redux Toolkit managed the application's global state. React Router DOM handled client-side navigation, allowing seamless transitions between application pages without full-page reloads.

Table 4.1: Frontend Development Tools

Tool / Technology	Purpose
React.js	Building reusable user interface components
Vite	Frontend build tool and development server
JavaScript (ES6+)	Client-side programming language
Tailwind CSS	Responsive user interface design
Redux Toolkit	Global state management
React Router DOM	Client-side routing and navigation
Axios	HTTP communication with the backend
Clerk Authentication	User authentication and session management
Lucide React	User interface icons
React Hot Toast	Notification messages
CSS3	Custom styling and animations
HTML5	User interface structure

Table 30 - Frontend Development Tools

4.2.2 Backend Development Tools

The backend of **UniSphere** was implemented using Node.js and Express.js following the MVC architectural pattern. The backend is responsible for processing client requests, managing business logic, communicating with the

MongoDB database, handling authentication tokens, and providing RESTful APIs. Additional middleware and supporting libraries were integrated to improve security, request validation, and file handling.

Table 4.2: Backend Development Tools

Tool / Technology	Purpose
Node.js	Server-side runtime environment
Express.js	RESTful API development
JavaScript (ES6+)	Backend programming language
Mongoose	MongoDB Object Data Modeling (ODM)
Clerk SDK	Authentication verification
JWT	Secure authentication tokens
Multer	File upload handling
CORS	Cross-Origin Resource Sharing
Dotenv	Environment variable management
Server-Sent Events (SSE)	Real-time notifications and messaging

Table 31 - Backend Development Tools

4.2.3 Database Tools

MongoDB was selected as the primary database management system because of its document-oriented architecture and seamless integration with the MERN Stack. The database stores all system information, including user profiles, academic posts, comments, messages, notifications, reports, and verification requests. Mongoose was used to define schemas, manage document relationships, and simplify interactions with the database.

Table 4.3: Database Tools

Tool / Technology	Purpose
MongoDB	NoSQL document database
MongoDB Atlas	Database hosting
Mongoose	Object Data Modeling (ODM)

Table 32 - Database Tools

4.2.4 Summary of Development Tools

The **UniSphere** platform integrates a comprehensive collection of modern web development technologies. React.js and Tailwind CSS provide a responsive and interactive frontend, while Node.js and Express.js implement the server-side business logic through RESTful APIs. MongoDB and Mongoose manage persistent data storage efficiently, and Clerk Authentication ensures secure identity management. Additional libraries such as Redux Toolkit, Axios, React Router DOM, ImageKit, and Server-Sent Events enhance the application's functionality, usability, and overall performance.

4.3 System Interfaces

Figure 4.1: Login Interface of the UniSphere Platform

Figure 4.1 presents the login interface of the UniSphere platform, which serves as the main entry point for users to access the system. The interface provides two authentication methods: traditional email and password login, and Google Single Sign-On (SSO) using Clerk Authentication. The page features a responsive and modern design developed with React.js and Tailwind CSS, ensuring an intuitive user experience across different devices. Additionally, options such as password visibility, "Remember Me," and password recovery improve usability and accessibility while maintaining secure authentication.

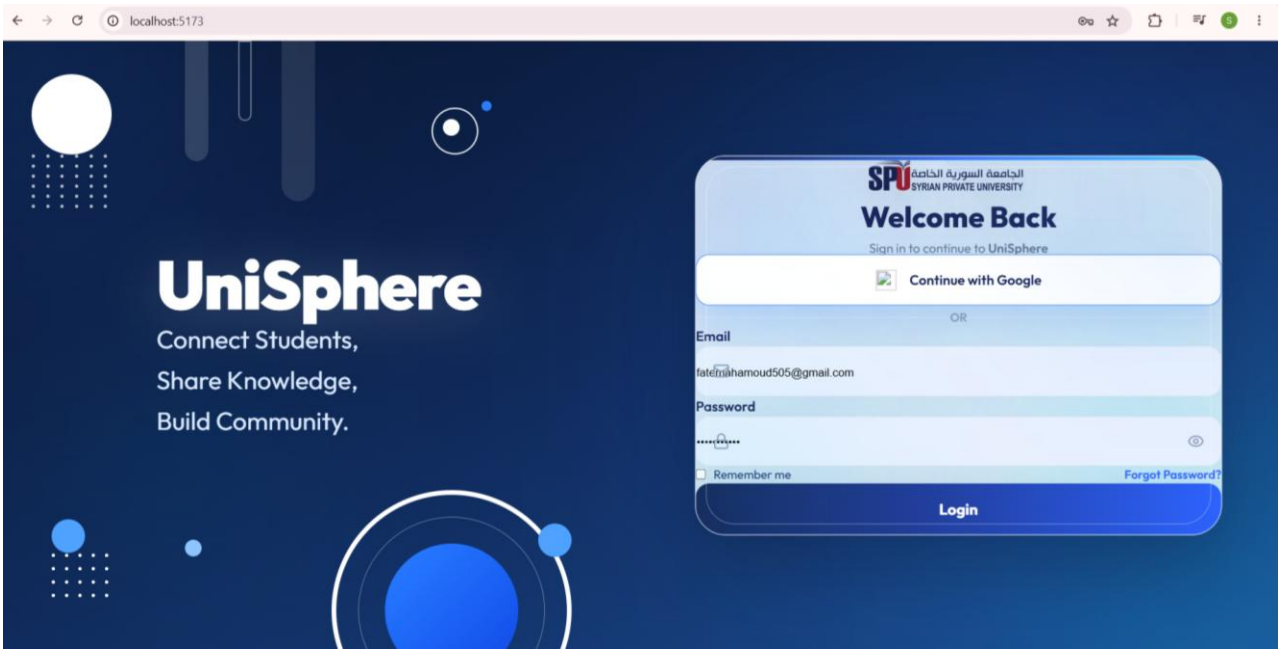

Figure 51 - Login Interface

4.3.2 Faculty Selection Interface

Figure 4.2: Faculty Selection Interface of the UniSphere Platform

Figure 52 - Faculty Selection Interface

Figure 4.2 illustrates the faculty selection interface, which is displayed to users during their first login after successful authentication. This interface enables users

to select their academic faculty before accessing the platform's main features. Each faculty is represented by a dedicated interactive card containing its official name and logo, providing a clear and user-friendly selection process. Once a faculty is selected, the information is stored in the user's profile and used to personalize the academic experience and organize users according to their respective colleges.

4.3.3 Account Verification Interface

Figure 4.3: Account Verification Interface of the UniSphere Platform

Figure 53 - Account Verification Interface

Figure 4.3 presents the account verification interface, which is displayed after the faculty selection process. This page allows users to choose the appropriate verification method based on their role as either a **Student** or a **Professor**. The interface separates the verification process into two distinct options, ensuring that each user follows the appropriate validation procedure before accessing the platform. This step enhances the security and credibility of the system by ensuring that only verified members of the university community can use UniSphere's academic services.

4.3.4 Student Verification Interface

Figure 4.4: Student Verification Interface of the UniSphere Platform

Figure 54 - Student Verification Interface

Figure 4.4 illustrates the student verification interface, where students are required to enter their official university identification number to verify their academic identity. The interface includes a simple input field and a submission button that sends the entered information to the backend for validation. This verification step ensures that only legitimate university students can access the platform's academic services. Upon successful verification, the user's account status is updated, allowing access to the remaining features of the UniSphere platform.

4.3.5 Professor Verification Interface

Figure 4.5: Professor Verification Interface of the UniSphere Platform

Figure 55 - Professor Verification Interface

Figure 4.5 illustrates the professor verification interface, which is designed specifically for faculty members to verify their academic identity before accessing the platform. Professors are required to enter their full official name, which is validated against the university's records through the backend verification process. This mechanism ensures that only authorized academic staff obtain professor privileges within the system. Successful verification enables professors to access features dedicated to teaching staff while maintaining the security and integrity of the UniSphere platform.

4.3.6 Feed Interface

Figure 4.6: Feed Interface of the UniSphere Platform

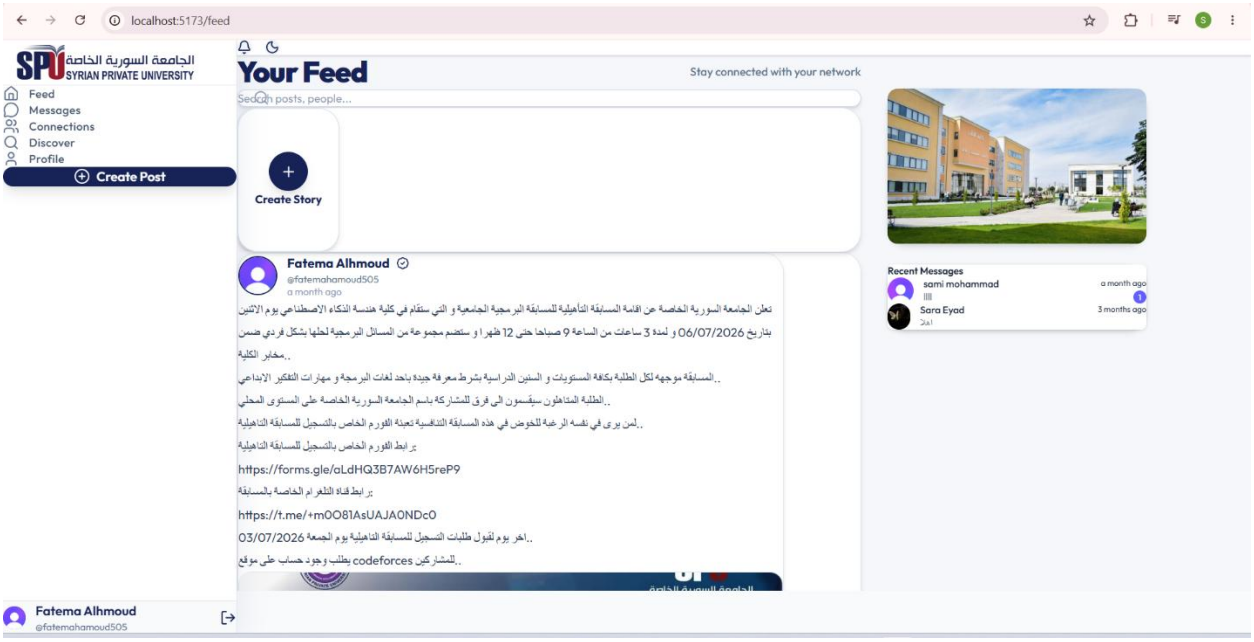

Figure 56 - Feed Interface of the UniSphere Platform

Figure 4.6 illustrates the main feed interface of the UniSphere platform, which serves as the central hub for academic interaction among users. The interface displays posts shared by students and professors in chronological order, allowing users to browse academic announcements, educational content, and discussions. It also provides quick access to story creation, recent conversations, and navigation menus for the platform's primary modules. The feed is designed to promote collaboration and knowledge sharing through an organized and responsive user interface that supports daily academic communication.

4.3.7 Messages Interface

Figure 4.7: Messages Interface of the UniSphere Platform

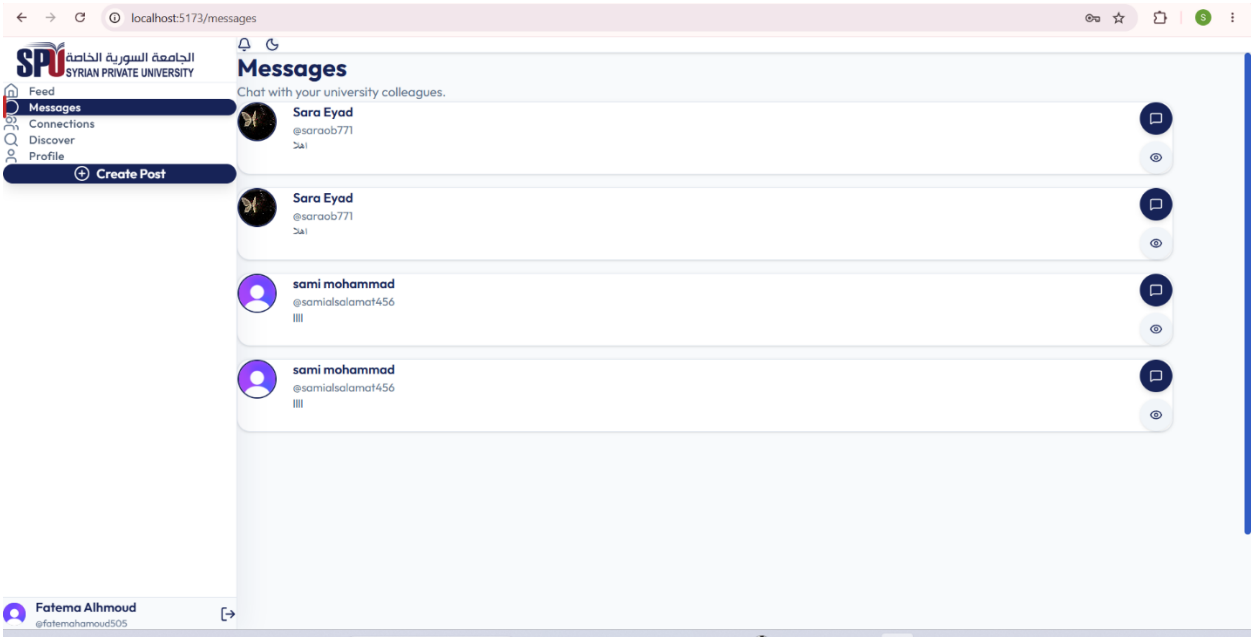

Figure 57 - Messages Interface

Figure 4.7 illustrates the messages interface, which displays the user's active conversations with other members of the UniSphere platform. The interface presents a list of connected users along with their profile pictures, usernames, and the most recent exchanged messages to facilitate quick communication. Users can select any conversation to open the corresponding chat window and continue real-time messaging. This interface enhances collaboration and academic communication by providing an organized and user-friendly messaging environment.

4.3.8 Connections Interface

Figure 4.8: Connections Interface of the UniSphere Platform

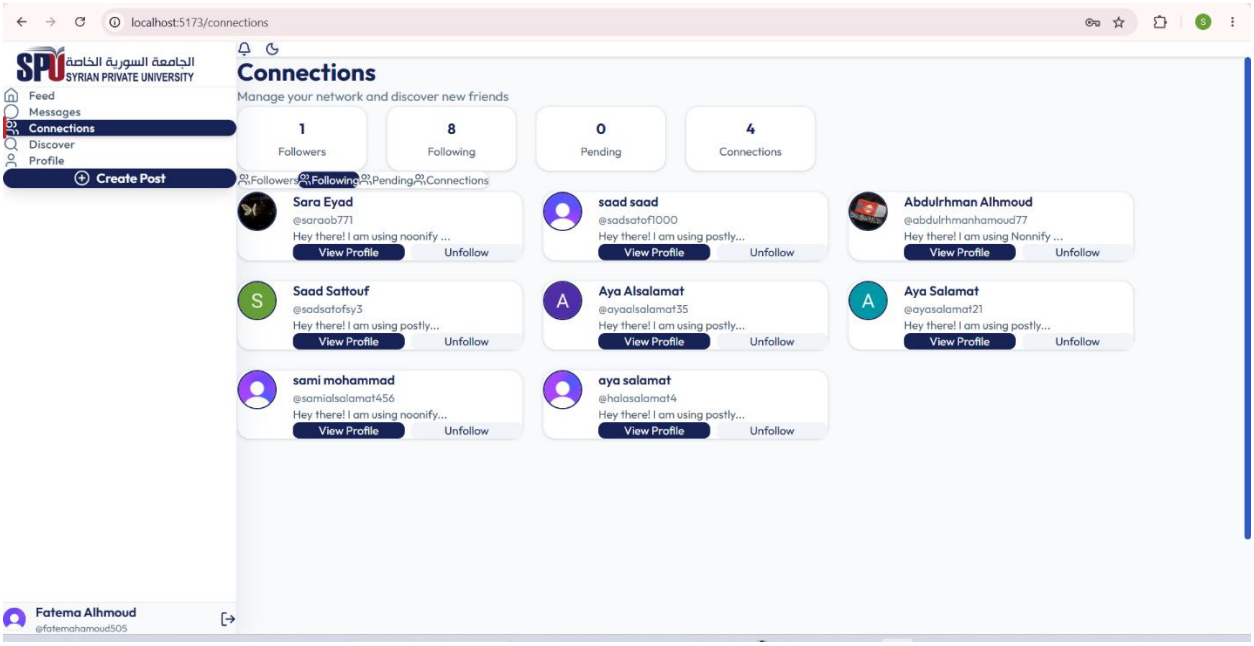

Figure 58 - Connections Interface

Figure 4.8 presents the connections interface, which enables users to manage their academic network within the UniSphere platform. The page displays connection statistics, including followers, following, pending requests, and established connections, providing users with an overview of their social interactions. It also allows users to view profiles, send or remove connections, and manage existing relationships through a simple and organized interface. This functionality encourages collaboration among students and professors by facilitating the creation and management of academic connections.

4.3.9 Discover Interface

Figure 4.9: Discover Interface of the UniSphere Platform

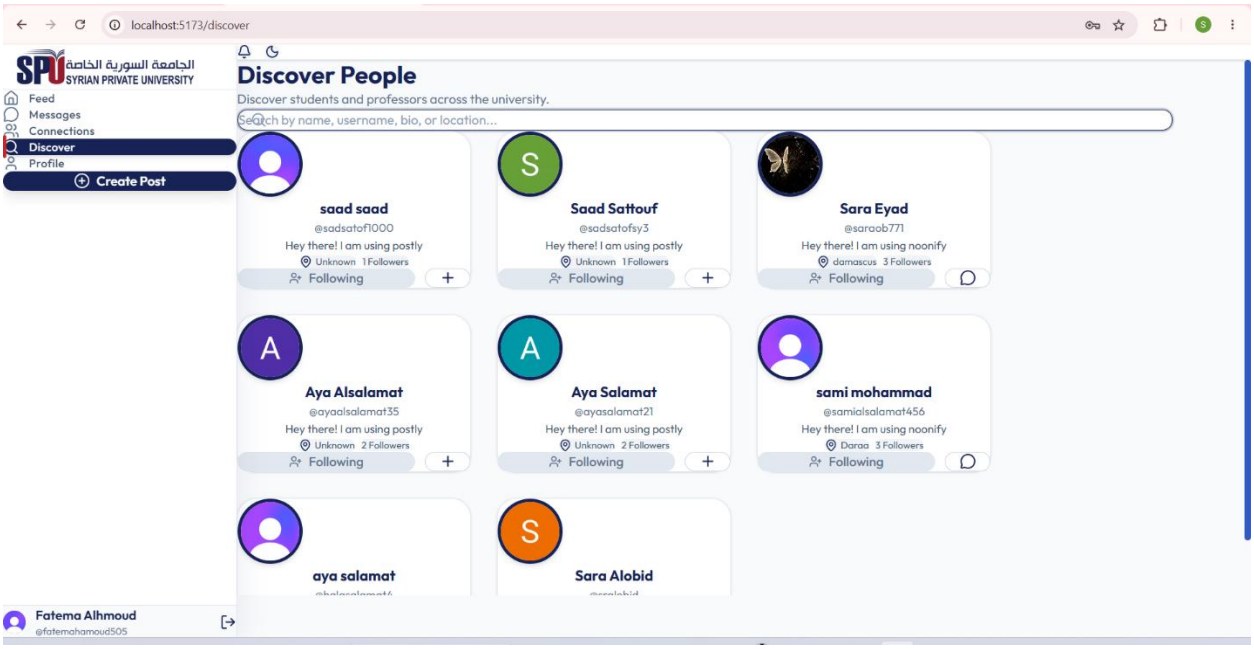

Figure 59 - Discover Interface

Figure 4.9 illustrates the discover interface, which enables users to search for and explore students and professors across the university community. The interface provides a search bar for locating users by name, username, biography, or location, and displays matching profiles in an organized card-based layout. Each profile includes basic user information and options to view the profile or send a connection request. This feature promotes academic networking by helping users discover new colleagues and expand their professional and educational connections within the UniSphere platform.

4.3.10 Profile Interface

Figure 4.10: User Profile Interface of the UniSphere Platform

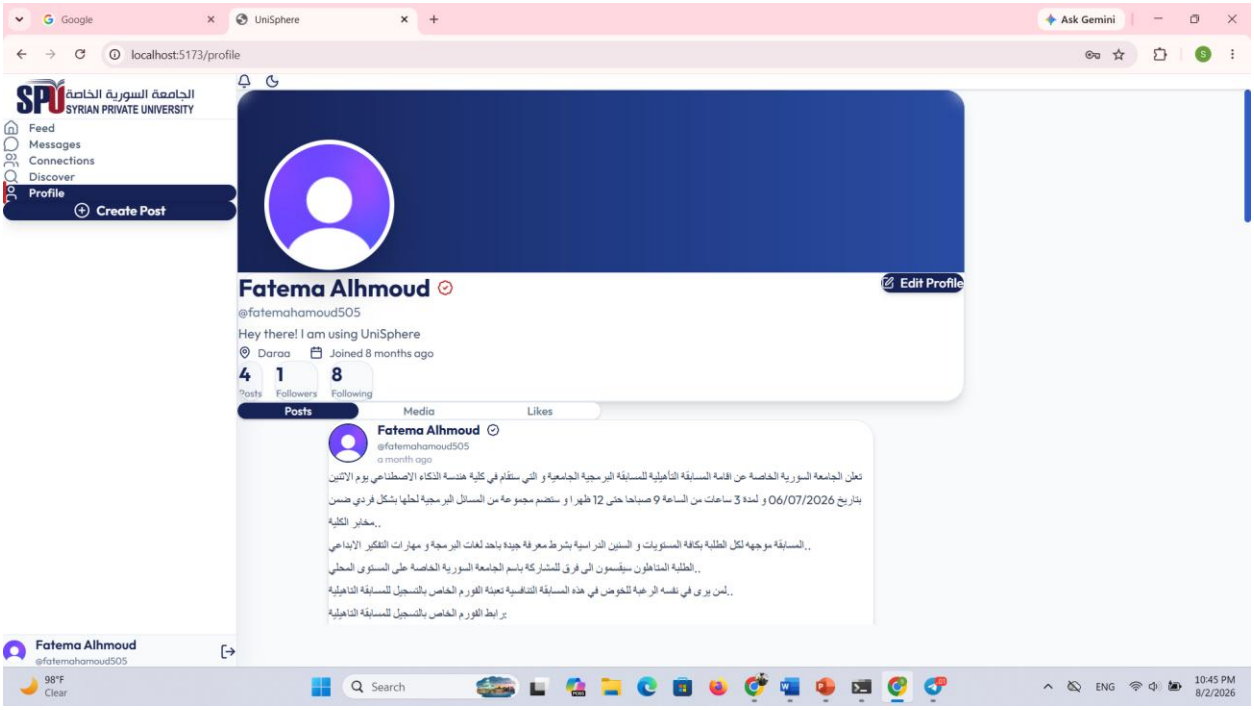

Figure 60 - User Profile Interface

Figure 4.10 presents the user profile interface, which displays the personal and academic information of a registered user within the UniSphere platform. The page includes profile details such as the user's name, username, biography, location, and social statistics, including the number of posts, followers, and following. Users can also edit their profile information and browse their published posts, media, and liked content through dedicated tabs. This interface provides a centralized view of the user's academic identity while supporting profile management and social interaction within the platform.

4.3.11 Notifications Interface

Figure 4.11: Notifications Interface of the UniSphere Platform

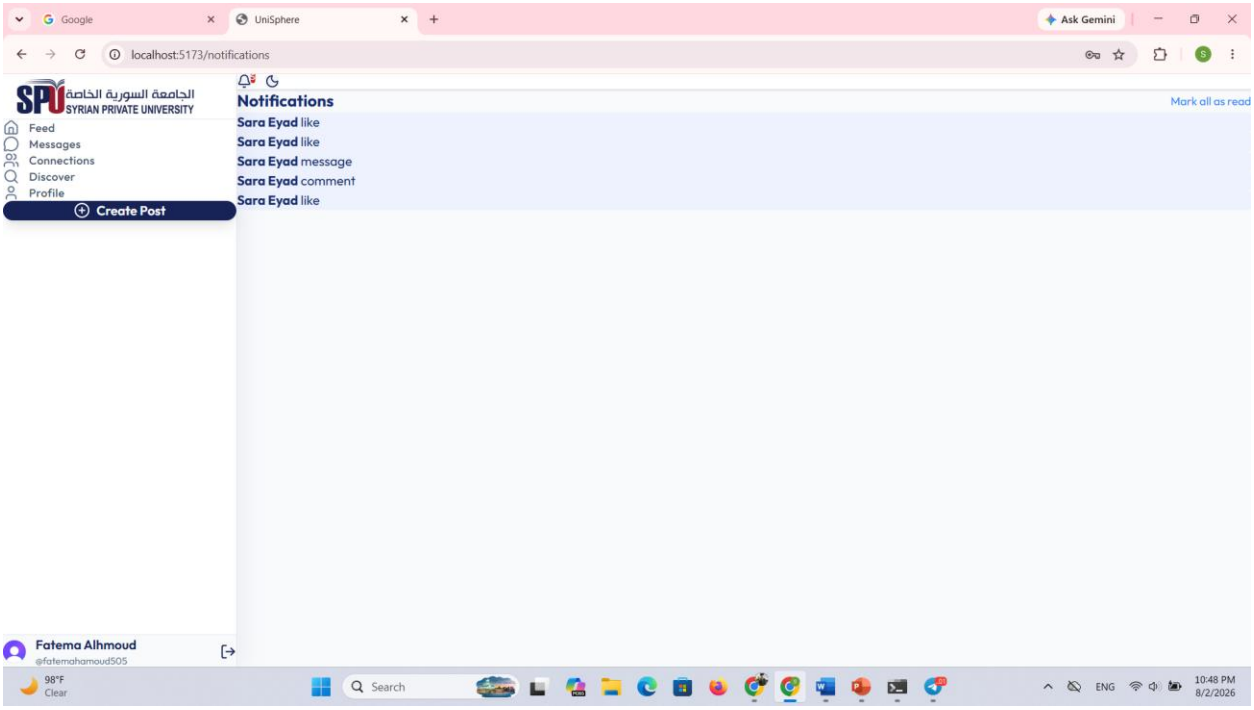

Figure 61 - Notifications Interface

Figure 4.11 illustrates the notifications interface, which provides users with real-time updates regarding activities performed within the UniSphere platform. The interface displays notifications generated from user interactions such as likes, comments, messages, and other system events in a chronological list. It also includes an option to mark all notifications as read, allowing users to efficiently manage their notification history. This feature improves user engagement by ensuring that important academic and social interactions are delivered promptly and remain easily accessible.

4.3.12 Create Post Interface

Figure 4.12: Create Post Interface of the UniSphere Platform

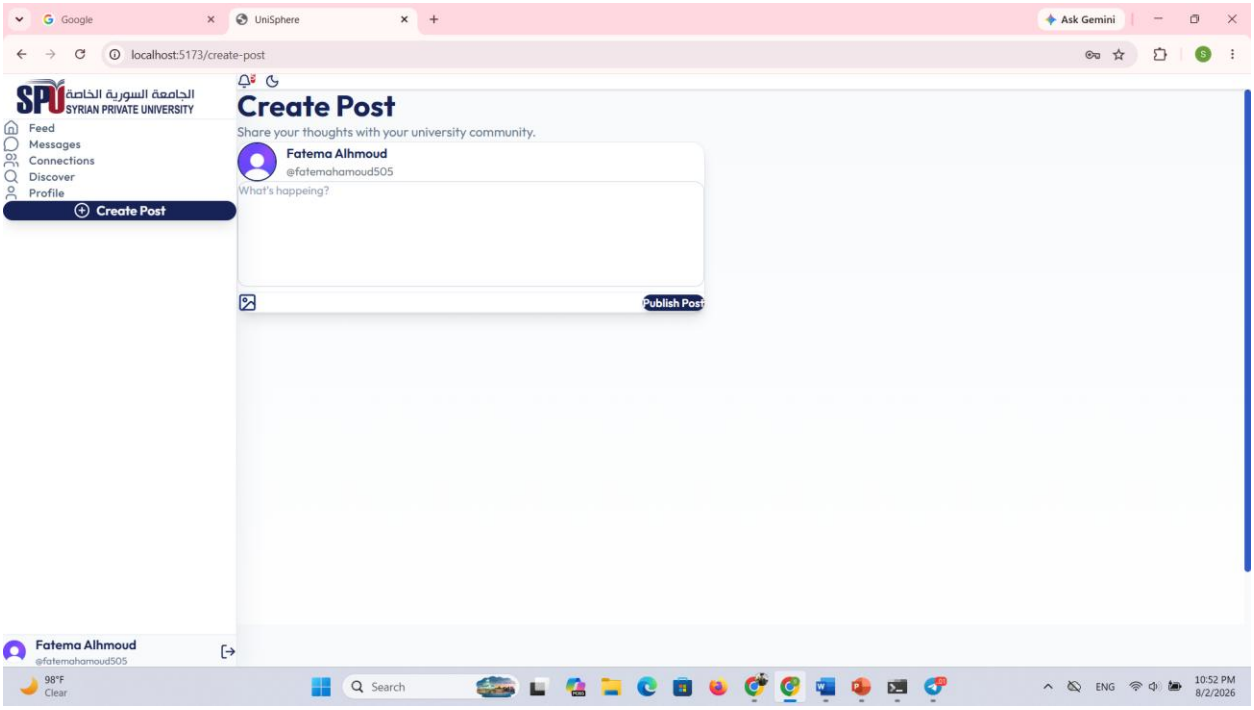

Figure 62 - Create Post Interface

Figure 4.12 illustrates the post creation interface, which enables users to publish academic content within the UniSphere community. The interface allows users to enter textual content, attach images, and submit posts through a simple and responsive form. After submission, the post is stored in the database and immediately appears in the news feed for other users. This feature supports knowledge sharing and encourages academic discussions among students and professors.

4.3.13 Create Story Interface

Figure 4.13: Create Story Interface of the UniSphere Platform

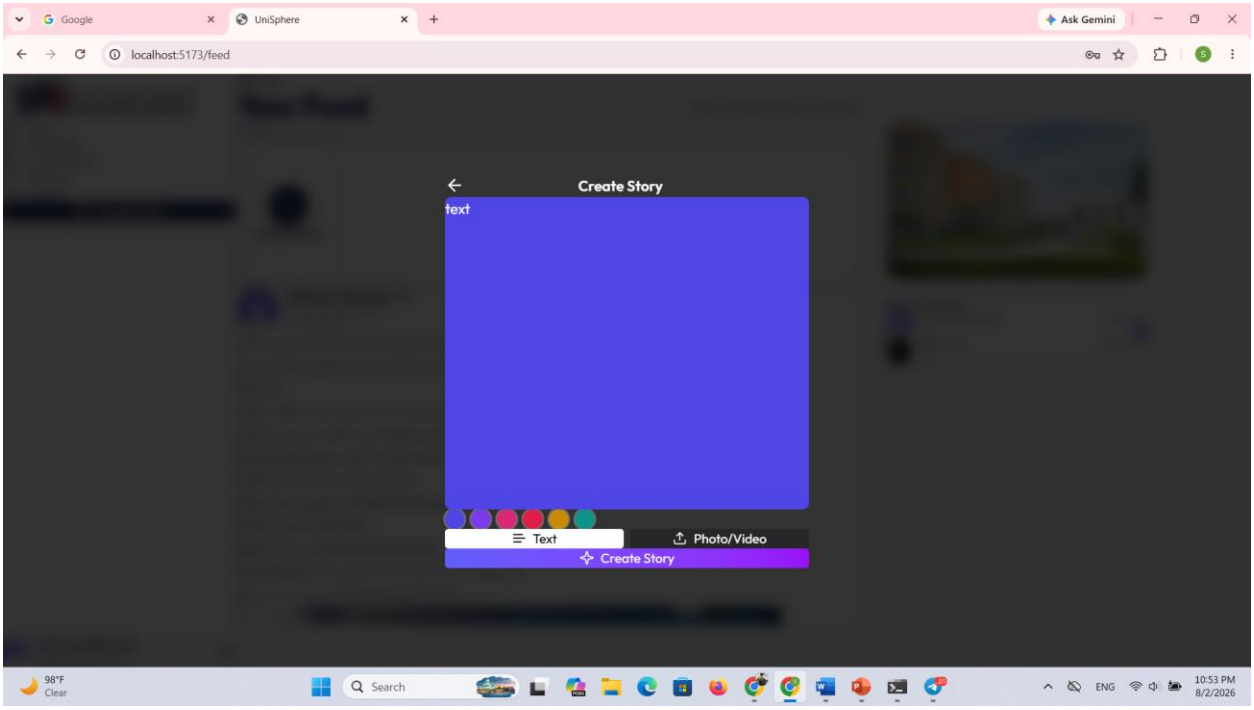

Figure 63 - Create Story Interface

Figure 4.13 presents the story creation interface, which enables users to share temporary content with other members of the platform. Users can create text-based stories by selecting different background colors or upload images and videos to personalize their stories. The interface provides an intuitive editing environment before publishing the story to the community. This functionality enhances user engagement by supporting quick and interactive content sharing in addition to traditional academic posts.

4.3.14 Chat Interface

Figure 4.14: Real-Time Chat Interface of the UniSphere Platform

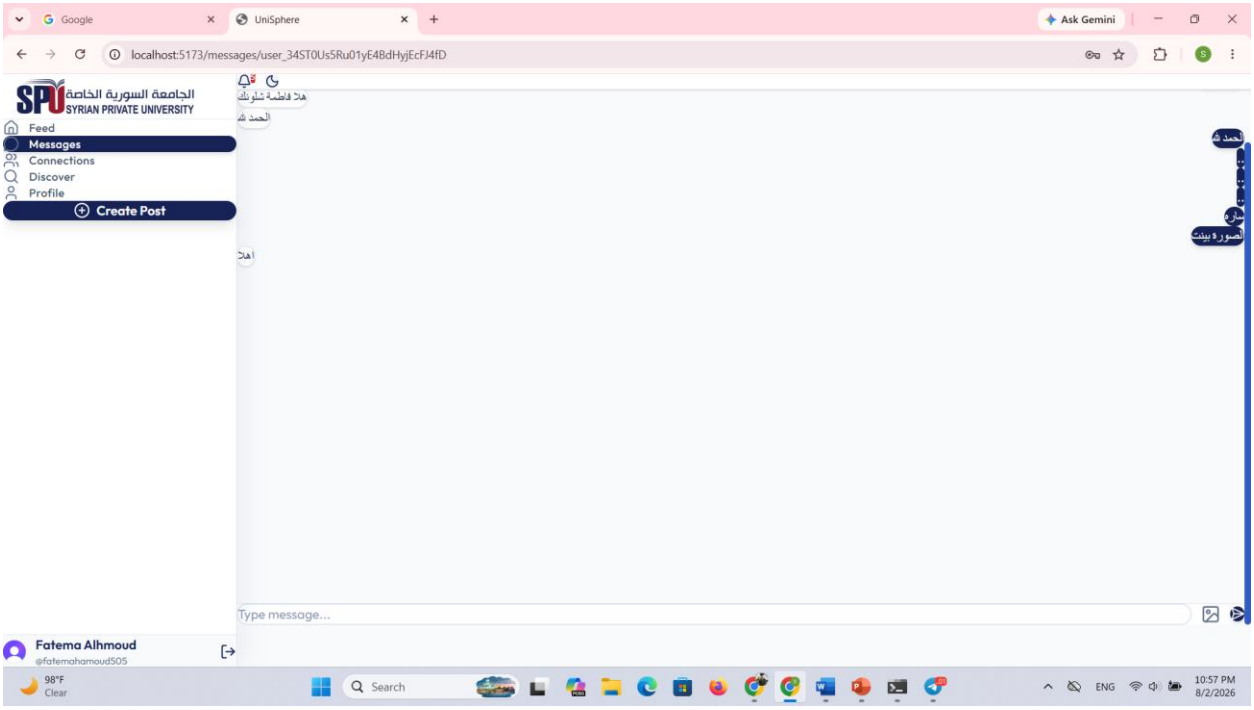

Figure 64 - Real-Time Chat Interface

Figure 4.14 illustrates the real-time chat interface, which enables direct communication between connected users within the UniSphere platform. The interface displays the conversation history in chronological order and distinguishes sent and received messages to improve readability. Users can compose text messages, attach images, and send them instantly through the integrated real-time messaging service. This functionality facilitates efficient academic collaboration and supports immediate communication between students and professors in a secure and user-friendly environment.

4.3.15 Edit Profile Interface

Figure 4.15: Edit Profile Interface of the UniSphere Platform

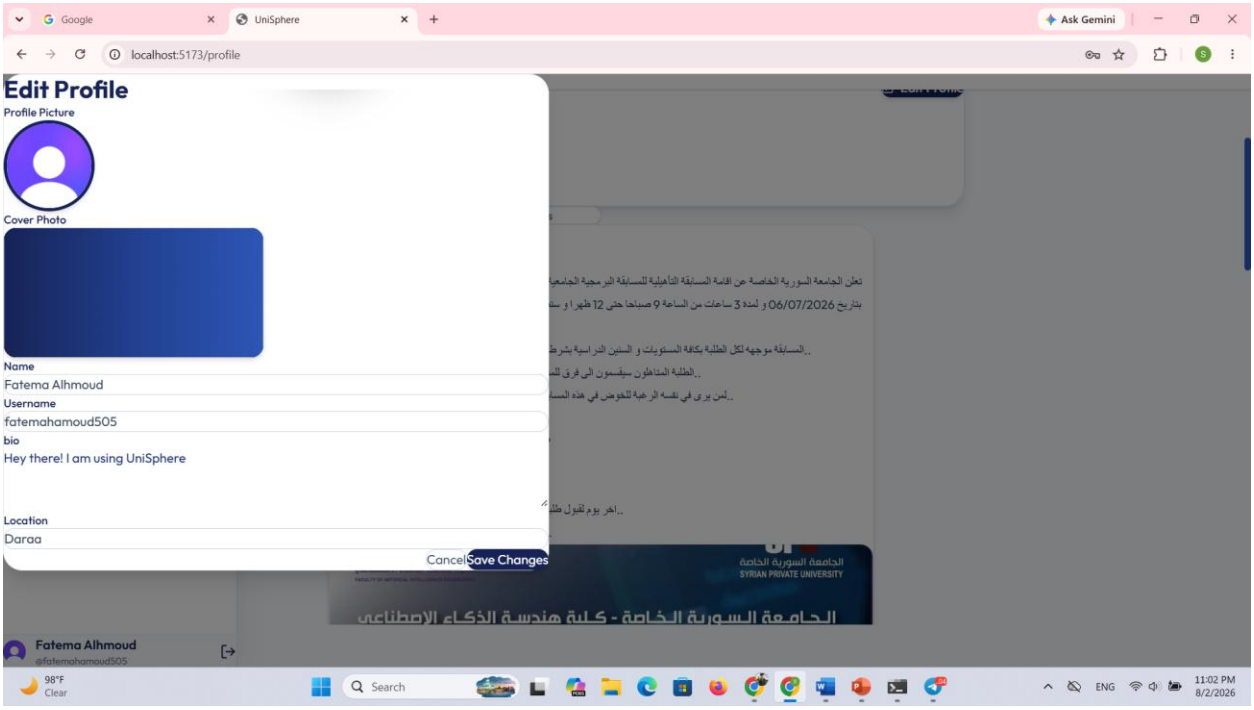

Figure 65 - Edit Profile Interface

Figure 4.15 illustrates the profile editing interface, which allows users to update their personal and academic information within the UniSphere platform. The interface provides editable fields for the user's name, username, biography, and location, in addition to options for changing the profile picture and cover photo. After modifying the desired information, users can save the changes, which are immediately reflected across the platform. This feature enables users to maintain an accurate and personalized academic profile while improving their visibility within the university community.

4.3.16 Admin Reports Interface

Figure 4.16: Admin Reports Interface of the UniSphere Platform

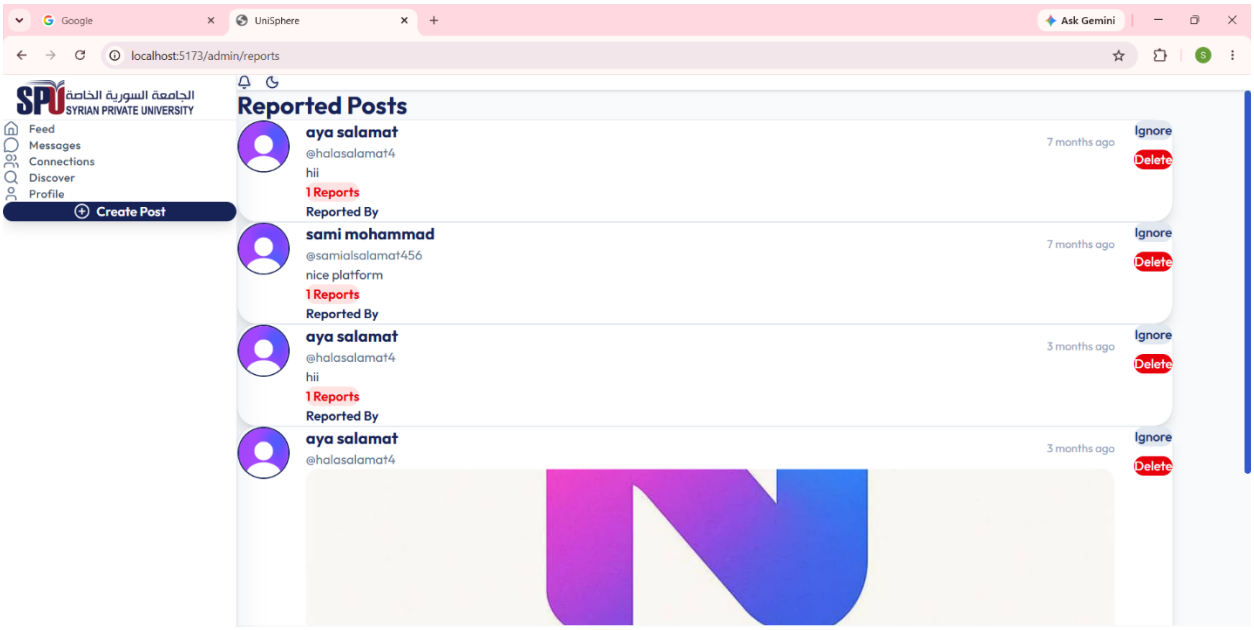

Figure 66 - Admin Reports Interface

Figure 67 - Admin Reports Interface

Figure 4.16 illustrates the administrative reports interface, which is designed to assist administrators in moderating content published on the UniSphere platform. The interface displays all reported posts along with the report count, reporter information, publication date, and post content, allowing administrators to review each case efficiently. Administrators can either ignore invalid reports or permanently delete posts that violate the platform's policies. This moderation mechanism helps maintain a secure, respectful, and academically appropriate environment for all users by ensuring that inappropriate content is handled promptly.

4.4 Chapter Summary

This chapter presented the implementation phase of the UniSphere platform by describing the technologies, frameworks, and development tools used throughout the project. It explained the frontend and backend development environments, the database technologies, and the supporting tools that contributed to building the system. Furthermore, the chapter demonstrated the implementation of the platform through a collection of user interface screenshots, accompanied by detailed descriptions illustrating the functionality of each module, including authentication, verification, feed management, messaging, networking, profile management, notifications, content creation, and administrative moderation. These implementation results confirm that the developed system successfully satisfies the functional and non-functional requirements identified during the analysis and design phases, providing a secure, responsive, and user-friendly academic community platform.

Chapter Five Conclusion and Future Work

5.1 Conclusion

This project presented the design and implementation of **UniSphere**, an academic community platform developed to enhance communication and collaboration among students and professors within the university environment. The platform was built using a modern full-stack architecture based on **React.js**, **Node.js**, **Express.js**, and **MongoDB**, while **Clerk Authentication** was integrated to provide secure user authentication and account management. The system follows the **Client–Server** architecture combined with the **MVC** design pattern, resulting in a scalable, maintainable, and modular software solution.

Throughout the development process, the platform successfully implemented all planned functional requirements, including secure authentication, faculty selection, account verification, post and story publishing, real-time messaging, notifications, profile management, user discovery, connection management, and administrative report moderation. The use of Scrum methodology enabled incremental development, continuous testing, and regular improvements, contributing to a reliable and user-friendly application.

Overall, the developed platform demonstrates that modern web technologies can effectively support academic communication by providing a secure, interactive, and responsive environment. UniSphere offers a practical solution for strengthening collaboration and knowledge sharing among university members while providing a solid foundation for future enhancements.

5.2 Future Work

Although UniSphere successfully achieves the objectives defined in this project, several improvements can be implemented in future versions to further enhance its functionality and user experience. Potential future developments include:

- Developing dedicated **Android** and **iOS** mobile applications.
- Integrating **AI-based recommendation systems** to suggest relevant academic content and potential connections.

- Supporting **video conferencing** and virtual classroom sessions.
- Adding **group chats** and collaborative discussion communities.
- Implementing **event management** and university calendar integration.
- Providing **file and document sharing** with version management.
- Introducing **real-time voice and video communication** between users.
- Supporting **multiple universities** within a single platform.
- Adding advanced **analytics dashboards** for administrators and university management.
- Enhancing platform security by integrating additional monitoring and threat detection mechanisms.

References

[1] React Documentation, "React," Available: https://react.dev/

[2] Node.js Documentation, "Node.js," Available: https://nodejs.org/

[3] Express.js Documentation, "Express - Fast, Unopinionated, Minimalist Web Framework," Available: https://expressjs.com/

[4] MongoDB Documentation, "MongoDB Manual," Available: https://www.mongodb.com/docs/

[5] Mongoose Documentation, "Mongoose ODM," Available: https://mongoosejs.com/

[6] Clerk Documentation, "Clerk Authentication," Available: https://clerk.com/docs

[7] Redux Toolkit Documentation, "Redux Toolkit," Available: https://redux-toolkit.js.org/

[8] React Router Documentation, "React Router," Available: https://reactrouter.com/

[9] Socket.io Documentation, "Socket.IO," Available: https://socket.io/

[10] MDN Web Docs, "JavaScript," Available: https://developer.mozilla.org/

[11] Tailwind CSS Documentation, "Tailwind CSS," Available: https://tailwindcss.com/

[12] Figma, "Collaborative Interface Design Tool," Available: https://www.figma.com/